

Distance to a Curve & Least-Squares Fitting

Part 1 finds the shortest distance from a point to a curve three ways: exactly with calculus, with Newton–Raphson, and with golden section search. Part 2 fits a line and a parabola to data by minimizing the mean squared error.

 Nicole Yu Submitted: September 29, 2026 Source code

PART ONE • BY HAND

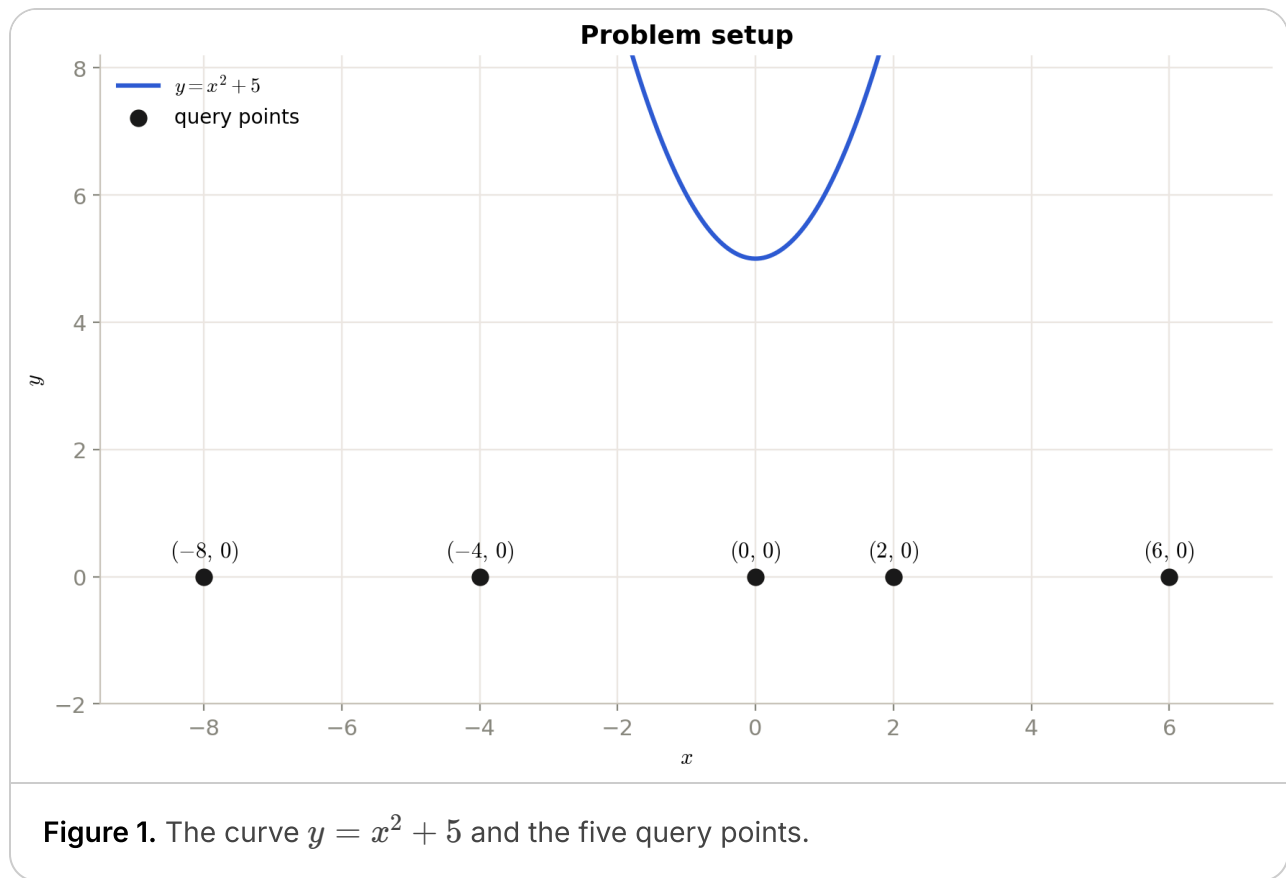
Shortest distance from a point to a function

1.1 Problem setup

We want the point on the curve $y = f(x)$ closest to a given point (x_0, y_0) , for $f(x) = x^2 + 5$ and the points $(0, 0)$, $(-4, 0)$, $(-8, 0)$, $(2, 0)$ and $(6, 0)$.

$$d = \sqrt{(x - x_0)^2 + (y - y_0)^2}$$
$$D(x) = d^2 = (x - x_0)^2 + (f(x) - y_0)^2$$

$D(x)$ is the squared length of the segment from (x_0, y_0) to the point $(x, f(x))$ on the curve. Since $d \geq 0$ and $\sqrt{\cdot}$ is increasing, the x that minimizes D also minimizes d , and D is easier to differentiate. At the minimum the segment is perpendicular to the curve's tangent.



EXACT

Analytical

Solve $D'(x) = 0$ in closed form.

ITERATIVE · USES D' , D''

Newton–Raphson

Follow the tangent of D' to its root.

ITERATIVE · USES D ONLY

Golden section

Shrink a bracket around the minimum.

1.2 Analytical solution

$$D'(x) = 2(x - x_0) + 2(f(x) - y_0)f'(x) = 0$$

with $f(x) = x^2 + 5$, $y_0 = 0$: $2x^3 + 11x - x_0 = 0$

For $f(x) = x^2 + 5$ and $y_0 = 0$, the condition is a cubic in x . For $(0, 0)$ it factors as $x(2x^2 + 11) = 0$, and since $2x^2 + 11 > 0$ the only root is $x = 0$. For the other points I divided by 2 to get $x^3 + px +$

$q = 0$ with $p = \frac{11}{2}$, $q = -\frac{x_0}{2}$, and used Cardano's formula:

$$x = \sqrt[3]{-\frac{q}{2} + \sqrt{\frac{q^2}{4} + \frac{p^3}{27}}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\frac{q^2}{4} + \frac{p^3}{27}}}$$

Because $p > 0$, each cubic has exactly one real root, so that critical point is the minimum. Plugging x^* back into d gives:

Point	x^*	Closest point	d
(0, 0)	0	(0, 5)	5
(-4, 0)	-0.356	(-0.356, 5.126)	6.290
(-8, 0)	-0.672	(-0.672, 5.452)	9.133
(2, 0)	0.181	(0.181, 5.033)	5.351
(6, 0)	0.520	(0.520, 5.270)	7.603

PART ONE: DISTANCE TO FUNCTION.

given $y = x^2 + 5$ and the following 5 points: $(0,0)$, $(-4,0)$, $(-8,0)$, $(2,0)$, $(6,0)$

i) Solve each point to f using analytical approach (by hand)

i) $(0,0)$ to $f(x) = x^2 + 5$

$$D(x) = (x - x_0)^2 + (f(x) - y_0)^2 \quad \leftarrow \text{Squared distance function}$$

$$\begin{aligned} \Rightarrow D'(x) &= 2(x - x_0) + 2(f(x) - y_0)f'(x) \\ &= 2(x - 0) + 2(x^2 + 5 - 0)(2x) \\ &= 2x + 2(x^2 + 5)(2x) \end{aligned}$$

#solve for x . $D'(x) = 0$

$$\begin{aligned} 0 &= x + (x^2 + 5)(2x) \\ &= x + 2x^3 + 10x \\ &= 2x^3 + 11x \\ &= x(2x^2 + 11) \end{aligned}$$

$\Rightarrow x = 0$ since $2x^2 + 11 > 0$ for all $x \in \mathbb{R}$

$$\begin{aligned} D(0) &= \sqrt{(0 - 0)^2 + (5 - 0)^2} \\ &= \sqrt{25} \\ &= 5 \end{aligned}$$

$\therefore$ minimum $d = 5$ at closest point $(0, 5)$

ii) $(-4,0)$ to $f(x) = x^2 + 5$

$$\begin{aligned} D'(x) &= 2(x - x_0) + 2(f(x) - y_0)f'(x) \\ &= 2(x - (-4)) + 2(x^2 + 5 - 0)(2x) \end{aligned}$$

#Set $D'(x) = 0$

$$\begin{aligned} 0 &= (x + 4) + (x^2 + 5)(2x) \\ &= 2x^3 + 11x + 4 \end{aligned}$$

*solve for x : use Cardano's formula (find roots of cubic eqn)

$$x^3 + \frac{11}{2}x + 2 = 0, \quad p = \frac{11}{2}, \quad q = 2$$

$$\begin{aligned} x &= \sqrt[3]{-\frac{2}{2} + \sqrt{\frac{2^2}{4} + \frac{(\frac{11}{2})^3}{27}}} + \sqrt[3]{-\frac{2}{2} - \sqrt{\frac{2^2}{4} + \frac{(\frac{11}{2})^3}{27}}} \\ &= \sqrt[3]{1.676} + \sqrt[3]{-3.676} \\ &= -0.3555 \end{aligned}$$

so closest point is $(-0.356, f(-0.356)) \Rightarrow (-0.356, 5.126)$

and distance to closest point is:

$$\begin{aligned} d &= \sqrt{((-0.356 + 4)^2 + (5.126 - 0)^2)} \\ &= 6.289 \end{aligned}$$

$\therefore$ at closest point $(-0.356, 5.126)$ the minimum distance is $d = 6.289$

iii) $(-8,0)$

#Set $D'(x) = 0$

$$\begin{aligned} 0 &= (x + 8) + (x^2 + 5)(2x) \\ &= 2x^3 + 11x + 8 \end{aligned}$$

*solve for x : $x^3 + \frac{11}{2}x + 4 = 0$, $p = \frac{11}{2}$, $q = 4$

$$x = \sqrt[3]{-2 + \sqrt{4 + \frac{(\frac{11}{2})^3}{27}}} + \sqrt[3]{-2 - \sqrt{4 + \frac{(\frac{11}{2})^3}{27}}}$$

$\underbrace{\hspace{1.5cm}}_{3.1878} \quad \underbrace{\hspace{1.5cm}}_{3.1878}$

$$= 1.059 - 1.731$$

$$= -0.672$$

so the closest point is $(-0.672, 5.452)$

and distance to closest point is:

$$d = \sqrt{((-0.672+8)^2 + (5.452-0)^2)}$$

$$= 9.134$$

$\therefore$ at closest point $(-0.672, 5.452)$ the minimum distance is $d = 9.134$

iv) $(2,0)$

Set $D'(x) = 0$

$$0 = (x-2) + (x^2+5)(2x)$$

$$= 2x^3 + 11x - 2$$

* solve for x : $x^3 + \frac{11}{2}x - 1 = 0$, $p = \frac{11}{2}$, $q = -1$

$$x = \sqrt[3]{\frac{1}{2} + \sqrt{\frac{1}{4} + \frac{(\frac{11}{2})^3}{27}}} + \sqrt[3]{\frac{1}{2} - \sqrt{\frac{1}{4} + \frac{(\frac{11}{2})^3}{27}}}$$

$$= 1.447 - 1.267$$

$$= 0.180$$

so the closest point is $(0.180, 5.033)$

and distance to closest point is:

$$d = \sqrt{((0.180-2)^2 + (5.033-0)^2)}$$

$$= 5.351$$

$\therefore$ at closest point $(0.180, 5.033)$ the minimum distance is $d = 5.351$

v) $(6,0)$

Set $D'(x) = 0$

$$0 = (x-6) + (x^2+5)(2x)$$

$$= 2x^3 + 11x - 6$$

* solve for x : $x^3 + \frac{11}{2}x - 3 = 0$, $p = \frac{11}{2}$, $q = -3$

$$x = \sqrt[3]{\frac{3}{2} + \sqrt{\frac{9}{4} + \frac{(\frac{11}{2})^3}{27}}} + \sqrt[3]{\frac{3}{2} - \sqrt{\frac{9}{4} + \frac{(\frac{11}{2})^3}{27}}}$$

$$= 1.639 - 1.119$$

$$= 0.520$$

so the closest point is $(0.520, 5.271)$

and distance to closest point is:

$$d = \sqrt{((0.520-6)^2 + (5.271-0)^2)}$$

$$= 7.604$$

$\therefore$ at closest point $(0.520, 5.271)$ the minimum distance is $d = 7.604$

1.3 Newton–Raphson by hand

$$x_{n+1} = x_n - \frac{D'(x_n)}{D''(x_n)}$$
$$D''(x) = 2 + 2f'(x)^2 + 2(f(x) - y_0)f''(x)$$

Newton–Raphson finds roots, so it is applied to $D'(x)$: the minimum of D is where $D' = 0$. Each step replaces D' with its tangent line at x_n and jumps to where that line is zero. Here $D'(x) = 2(2x^3 + 11x - x_0)$ and $D''(x) = 2(6x^2 + 11) > 0$, so D is convex and the root is the minimum.

I started from the vertex, $x = 0$. For $(0, 0)$ I started at $x = 1$, since 0 is already the answer. I did two iterations per point, tracking $|x_{n+1} - x_n|$ against the tolerance. After two steps every point agrees with the analytical answer to 2–3 decimals; the code continues until the step is below 10^{-7} .

2) Solve each point to f using Newton Raphson's method (algorithm) - 2 iterations

The algorithm:

i) select initial guess $f(x_0)$

ii) $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$ where $f(x_n)$ is the function you want root to. (in our case y')

i) $(0,0)$ to $y = x^2 + 5$

we want to minimize the square distance function. i.e. $D(x) = (x - x_0)^2 + (f(x) - y_0)^2$

$$\text{so: } D'(x) = 2x + 2(x^2 + 5)(2x)$$

$$= 2(2x^3 + 11x)$$

$$D''(x) = 2(6x^2 + 11)$$

ITERATION ONE: initial guess $x = 1$

$$D'(x) = 26, \quad D''(x) = 34$$

$$x_{n+1} = 1 - \frac{26}{34} = 0.2353$$

$$|x_{n+1} - x_n| \approx 0.765$$

ITERATION TWO: $x_n = 0.2353$

$$D'(x) = 5.2287, \quad D''(x) = 22.6644$$

$$x_{n+1} = 0.2353 - \left(\frac{5.2287}{22.6644} \right) = 4.599 \times 10^{-3}$$

$$|x_{n+1} - x_n| \approx 0.23$$

* we'd keep going until the $|x_{n+1} - x_n|$ value is below our tolerance but I'll save answer for algorithm section. Here I only demonstrate 2 iterations. However, notice the convergence towards $x = 0$.

if we stop @ iteration two:

$$\text{closest point} \rightarrow (0.004599, 5.00)$$

$$d = \sqrt{(0.004599)^2 + 5^2} \approx 5$$

} notice agreement w/ analytical solution above.

ii) $(-4,0)$ to $y = x^2 + 5$

we want to minimize the square distance function. i.e. $D(x) = (x - x_0)^2 + (f(x) - y_0)^2$

$$\text{so: } D'(x) = 2(2x^3 + 11x + 4)$$

$$D''(x) = 2(6x^2 + 11)$$

ITERATION ONE: initial guess $x = 0$

$$D'(x) = 8, \quad D''(x) = 22$$

$$x_{n+1} = 0 - \frac{8}{22} = -0.36$$

$$|x_{n+1} - x_n| \approx 0.36$$

ITERATION TWO: $x_n = -0.36$

$$D'(x) = -0.1915, \quad D''(x) = 23.586$$

$$x_{n+1} = -0.36 - \left(\frac{-0.1915}{23.586} \right) = -0.3555$$

$$|x_{n+1} - x_n| \approx 8.119 \times 10^{-3}$$

if we stop @ iteration two:

$$\text{closest point} \rightarrow (-0.36, 5.126)$$

$$d = \sqrt{(-0.36+4)^2 + (5.126)^2} \approx 6.29$$

} notice agreement w/ analytical solution above.

iii) $(-8, 0)$ to $y = x^2 + 5$

we want to minimize the square distance function. i.e. $D(x) = (x - x_0)^2 + (f(x) - y_0)^2$

so: $D'(x) = 2(2x^3 + 11x + 8)$

$D''(x) = 2(6x^2 + 11)$

ITERATION ONE: initial guess $x = 0$

$D'(x) = 16$, $D''(x) = 22$

$x_{n+1} = 0 - \frac{16}{22} = -0.72$

$|x_{n+1} - x_n| \approx 0.72$

ITERATION TWO: $x_n = -0.72$

$D'(x) = -1.5366$, $D''(x) = 28.3458$

$x_{n+1} = -0.72 - (-1.5366/28.3458)$
 $= -0.673$

$|x_{n+1} - x_n| \approx 0.054$

if we stop @ iteration two:

closest point $\rightarrow (-0.673, 5.453)$

$d = \sqrt{(-0.67+8)^2 + (5.45)^2} \approx 9.134$

} notice agreement w/ analytical solution above.

iv) $(2, 0)$ to $y = x^2 + 5$

we want to minimize the square distance function. i.e. $D(x) = (x - x_0)^2 + (f(x) - y_0)^2$

so: $D'(x) = 2(2x^3 + 11x - 2)$

$D''(x) = 2(6x^2 + 11)$

ITERATION ONE: initial guess $x = 0$

$D'(x) = -4$, $D''(x) = 22$

$x_{n+1} = 0 - \frac{(-4)}{22} = 0.18$

$|x_{n+1} - x_n| \approx 0.18$

ITERATION TWO: $x_n = 0.18$

$D'(x) = 0.0236$, $D''(x) = 22.396$

$x_{n+1} = 0.18 - (0.0236/22.396)$
 $= 0.181$

$|x_{n+1} - x_n| \approx 1 \times 10^{-3}$

if we stop @ iteration two:

closest point $\rightarrow (0.181, 5.033)$

$d = \sqrt{(0.181-2)^2 + (5.033)^2} \approx 5.35$

} notice agreement w/ analytical solution above.

v) $(6, 0)$ to $y = x^2 + 5$

$D'(x) = 2(2x^3 + 11x - 6)$

$D''(x) = 2(6x^2 + 11)$

ITERATION ONE: initial guess $x = 0$

$D'(x) = -12$, $D''(x) = 22$

$x_{n+1} = 0 - \frac{(-12)}{22} = 0.54$

ITERATION TWO: $x_n = 0.54$

$D'(x) = 0.648$, $D''(x) = 25.57$

$x_{n+1} = 0.54 - \frac{0.648}{25.57} = 0.520$

if we stop @ iteration two:

closest point $\rightarrow (0.520, 5.270)$

$d = \sqrt{(0.520-6)^2 + (5.27)^2} \approx 7.603$

} notice agreement w/ analytical solution above.

1.4 Golden section search by hand

$$\begin{aligned}x_1 &= a + (2 - \varphi)(b - a) \\x_2 &= b - (2 - \varphi)(b - a) \\ \varphi &= \frac{1 + \sqrt{5}}{2}\end{aligned}$$

Golden section search only compares values of $D(x)$, so it needs no derivatives. It keeps a bracket $[a, b]$ that contains the minimum and evaluates D at two interior points, 38.2% and 61.8% of the way across. The minimum cannot lie beyond the interior point with the larger D , so that end of the bracket is discarded. The bracket shrinks by a factor of 0.618 each step, and one interior point is reused, so each step needs only one new evaluation.

To pick the starting bracket I tabulated $D(x)$ between the vertex and the point and kept the interval around the smallest value. For example, for $(-4, 0)$: $D(0) = 41$, $D(-0.5) = 39.81$, $D(-1) = 45$, so $[-1, 0]$. For $(0, 0)$ I used $[-1, 1]$; because it is symmetric, the first comparison is a tie and either half can be kept. After two iterations the estimate is the midpoint of the bracket.

Note on notation in the hand work

In my hand calculations I labelled the two interior points the other way round from the standard formula above:

$$d = 0.618(b - a), \quad x_1 = a + d, \quad x_2 = b - d$$

Since $0.618 = \varphi - 1 = 1 - (2 - \varphi)$, these are the same two points, at 38.2% and 61.8% of the bracket, with x_1 and x_2 swapped. In my notation x_1 is the right point, so the rule reads: if $D(x_1) > D(x_2)$, discard $x > x_1$; otherwise discard $x < x_2$. The brackets and results are identical to the standard version.

3) Solve each point to f using Golden Section Search (algorithm) - 2 iterations

i) $d = GR \cdot (b - a)$ * where $GR \approx 0.618$

ii) $x_1 = a + d$

$x_2 = b - d$

iii) if $f(x_1) > f(x_2)$

• eliminate all $x > x_1$

• $x_1 \rightarrow b$, $x_2 \rightarrow x_1$

if $f(x_2) < f(x_1)$

• eliminate all $x < x_2$

• $x_2 \rightarrow a$, $x_1 \rightarrow x_2$

basic solving steps.

i) $(0, 0)$ to $y = x^2 + 5$

ITERATION ONE: $[-1, 1]$

using $[-1, 1]$ as $[a, b]$:

$d = 0.618 \cdot (1 - (-1)) = 1.236$

$x_1 = -1 + 1.236 = 0.236$

$x_2 = 1 - 1.236 = -0.236$

evaluate $D(x_1)$ and $D(x_2)$ where $D(x) = x^2 + (x^2 + 5)^2$ $\frac{(x-x_0)^2 + (f(x)-y_0)^2}{2}$

$D(0.236) = 0.236^2 + (0.236^2 + 5)^2 = 25.62$

$D(-0.236) = (-0.236)^2 + ((-0.236)^2 + 5)^2 = 25.62$

* this results in a tie. choose either interval for iteration two.

ITERATION TWO: $[-0.236, 1]$

$d = 0.618 \cdot (1 - (-0.236)) = 0.7638$

$x_1 = -0.236 + 0.7638 = 0.528$

$x_2 = 1 - 0.7638 = 0.236$

evaluate $D(x_1)$ and $D(x_2)$

$D(0.528) = 0.528^2 + (0.528^2 + 5)^2 = 28.14$

$D(0.236) = 0.236^2 + (0.236^2 + 5)^2 = 25.62$

$D(x_1) > D(x_2)$ so minimum is to the left.

New interval = $[-0.236, 0.528]$

so after 2 iterations, $x = (-0.236 + 0.528)/2 \approx 0.1459$

$y = 0.1459^2 + 5 \approx 5.021$

$d = \sqrt{0.1459^2 + 5.021^2} = 5.023$

ii) $(-4, 0)$ to $y = x^2 + 5$

tabulate $D(x)$ at a few values between vertex and points x coordinate to get an ideal interval to start iterating from.

x	0	-0.5	-1	-2
D(x)	41	39.81	45	85

minimum exists somewhere here.

ITERATION ONE: $[-1, 0]$:

$d = 0.618 \cdot (0 - (-1)) = 0.618$

$x_1 = a + d = -1 + 0.618 = -0.382$

$x_2 = b - d = 0 - 0.618 = -0.618$

evaluate $D(x_1)$ and $D(x_2)$

$D(-0.382) = (-0.382 + 4)^2 + ((-0.382)^2 + 5)^2 = 39.570$

$D(-0.618) = (-0.618 + 4)^2 + ((-0.618)^2 + 5)^2 = 40.403$

$D(x_1) < D(x_2)$ so minimum is on right

$$\text{new interval} = [-0.618, 0]$$

ITERATION TWO: $[-0.618, 0]$:

$$d = 0.618 \cdot (0 - (-0.618)) = 0.382$$

$$x_1 = -0.618 + 0.382 = -0.236$$

$$x_2 = 0 - 0.382 = -0.382$$

evaluate $D(x_1)$ and $D(x_2)$

$$D(-0.236) = (-0.236 + 4)^2 + ((-0.236)^2 + 5)^2 = 39.7278$$

$$D(-0.382) = (-0.382 + 4)^2 + ((-0.382)^2 + 5)^2 = 39.5704$$

$D(x_1) > D(x_2)$ so minimum is on right

$$\text{new interval} = [-0.618, -0.236]$$

so after 2 iterations, $x = (-0.618 + (-0.236)) / 2 \approx -0.427$

$$y = (-0.427)^2 + 5 \approx 5.182$$

$$d = \sqrt{(-0.427 + 4)^2 + 5.182^2} = 6.29$$

iii) $(-8, 0)$ to $y = x^2 + 5$

ITERATION ONE: $[-1, 0]$:

$$d = 0.618 \cdot (0 - (-1)) = 0.618$$

$$x_1 = a + d = -1 + 0.618 = -0.382$$

$$x_2 = b - d = 0 - 0.618 = -0.618$$

evaluate $D(x_1)$ and $D(x_2)$

$$D(-0.382) = (-0.382 + 8)^2 + ((-0.382)^2 + 5)^2 = 84.514$$

$$D(-0.618) = (-0.618 + 8)^2 + ((-0.618)^2 + 5)^2 = 83.459$$

$D(x_1) > D(x_2)$ so minimum is on right

$$\text{new interval} = [-1, -0.382]$$

ITERATION TWO: $[-1, -0.382]$:

$$d = 0.618 \cdot (-0.382 - (-1)) = 0.382$$

$$x_1 = -1 + 0.382 = -0.618$$

$$x_2 = -0.382 - 0.382 = -0.764$$

evaluate $D(x_1)$ and $D(x_2)$

$$D(-0.618) = (-0.618 + 8)^2 + ((-0.618)^2 + 5)^2 = 83.459$$

$$D(-0.764) = (-0.764 + 8)^2 + ((-0.764)^2 + 5)^2 = 83.537$$

$D(x_1) < D(x_2)$ so minimum is on left

$$\text{new interval} = [-0.764, -0.382]$$

so after 2 iterations, $x = (-0.764 + (-0.382)) / 2 \approx -0.573$

$$y = (-0.573)^2 + 5 \approx 5.328$$

$$d = \sqrt{(-0.573 + 8)^2 + 5.328^2} = 9.14$$

iv) $(2, 0)$ to $y = x^2 + 5$

ITERATION ONE: $[0, 1]$:

$$d = 0.618 \cdot (1 - 0) = 0.618$$

$$x_1 = a + d = 0 + 0.618 = 0.618$$

$$x_2 = b - d = 1 - 0.618 = 0.382$$

evaluate $D(x_1)$ and $D(x_2)$

$$D(0.618) = ((0.618) - 2)^2 + ((0.618)^2 + 5)^2 = 35.819$$

$$D(0.382) = ((0.382) - 2)^2 + ((0.382)^2 + 5)^2 = 29.098$$

$D(x_1) > D(x_2)$ so minimum is on right

$$\text{new interval} = [0, 0.618]$$

ITERATION TWO: $[0, 0.618]$

$$d = 0.618 \cdot (0.618 - 0) = 0.382$$

$$x_1 = 0 + 0.382 = 0.382$$

$$x_2 = 0.618 - 0.382 = 0.236$$

evaluate $D(x_1)$ and $D(x_2)$

$$D(0.382) = ((0.382) - 2)^2 + ((0.382)^2 + 5)^2 = 29.098$$

$$D(0.236) = ((0.236) - 2)^2 + ((0.236)^2 + 5)^2 = 28.672$$

$D(x_1) > D(x_2)$ so minimum is on left of interval

$$\text{new interval} = [0, 0.236]$$

so after 2 iterations, $x = (0 + 0.236)/2 \approx 0.118$

$$y = (0.118)^2 + 5 \approx 5.0139$$

$$d = \sqrt{(0.118 - 2)^2 + (5.0139)^2} \approx 5.355$$

v) $(6, 0)$ to $y = x^2 + 5$

ITERATION ONE: $[0, 1]$

$$d = 0.618 \cdot (1 - 0) = 0.618$$

$$x_1 = a + d = 0 + 0.618 = 0.618$$

$$x_2 = b - d = 1 - 0.618 = 0.382$$

evaluate $D(x_1)$ and $D(x_2)$

$$D(0.618) = ((0.618) - 6)^2 + ((0.618)^2 + 5)^2 = 57.931$$

$$D(0.382) = ((0.382) - 6)^2 + ((0.382)^2 + 5)^2 = 58.042$$

$D(x_1) < D(x_2)$ so minimum is on right

$$\text{new interval} = [0.382, 1]$$

ITERATION TWO: $[0.382, 1]$

$$d = 0.618 \cdot (1 - 0.382) = 0.382$$

$$x_1 = 0.382 + 0.382 = 0.764$$

$$x_2 = 1 - 0.382 = 0.618$$

evaluate $D(x_1)$ and $D(x_2)$

$$D(0.764) = ((0.764) - 6)^2 + ((0.764)^2 + 5)^2 = 58.593$$

$$D(0.618) = ((0.618) - 6)^2 + ((0.618)^2 + 5)^2 = 57.931$$

$D(x_1) > D(x_2)$ so minimum is on left

$$\text{new interval} = [0.382, 0.618]$$

so after 2 iterations, $x = (0.382 + 0.618)/2 \approx 0.5$

$$y = (0.5)^2 + 5 \approx 5.25$$

$$d = \sqrt{(0.5 - 6)^2 + (5.25)^2} \approx 7.603$$

Newton–Raphson and golden section in Python

1.5 Code

`distance.py` contains the handout's two solvers, changed only to also return their steps. It also has `choose_bracket`, which samples $D(x)$ and brackets the lowest sample (the same idea as the hand tabulation), and the driver. `plots.py` only draws. Run `python distance.py` from `A1/code/` for every case on this page, or `python distance.py a b c x0 y0` for any parabola $y = ax^2 + bx + c$ and point. Plots are saved to `A1/media/`.

distance.py: solvers & driver

Python

```
"""
SYDE 572 Assignment 1, Part 1: shortest distance from a point (x0, y0) to a curve y = f(x).

find_distance_newton() and golden_section_search() are the handout's algorithms,
changed only to also return their steps for plotting.

    python distance.py                # assignment parabola + other functions
    python distance.py a b c x0 y0    # any parabola y = ax^2 + bx + c and point
Plots are saved to ../media/
"""

import math
import os
import sys

import numpy as np

import plots

MEDIA = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "media")

def find_distance_newton(x0, y0, f, df, ddf, initial_guess=0.0, tolerance=1e-7,
max_iter=100):
    x = initial_guess
    history = [x]
    for _ in range(max_iter):
        D_prime = 2 * (x - x0) + 2 * (f(x) - y0) * df(x)
        D_double_prime = 2 + 2 * df(x)**2 + 2 * (f(x) - y0) * ddf(x)
        step = D_prime / D_double_prime
        x -= step
        history.append(x)
        if abs(step) < tolerance:
            break
```

```

return math.hypot(x - x0, f(x) - y0), x, history

def golden_section_search(x0, y0, f, a, b, tolerance=1e-7):
    resphi = 2 - (1 + math.sqrt(5)) / 2
    def dist_sq(x): return (x - x0)**2 + (f(x) - y0)**2
    x1, x2 = a + resphi * (b - a), b - resphi * (b - a)
    f_x1, f_x2 = dist_sq(x1), dist_sq(x2)
    brackets = [(a, b)]
    while abs(b - a) > tolerance:
        if f_x1 < f_x2:
            b, x2, f_x2 = x2, x1, f_x1
            x1 = a + resphi * (b - a)
            f_x1 = dist_sq(x1)
        else:
            a, x1, f_x1 = x1, x2, f_x2
            x2 = b - resphi * (b - a)
            f_x2 = dist_sq(x2)
        brackets.append((a, b))
    best_x = (a + b) / 2
    return math.sqrt(dist_sq(best_x)), best_x, brackets

def choose_bracket(x0, y0, f, half_width=10, n=81):
    """Sample D(x) on [x0 - half_width, x0 + half_width] and bracket the lowest sample
    with its two neighbours (the same idea as tabulating D(x) by hand)."""
    grid = np.linspace(x0 - half_width, x0 + half_width, n)
    i = int(np.nanargmin((grid - x0)**2 + (f(grid) - y0)**2))
    return grid[max(i - 1, 0)], grid[min(i + 1, n - 1)]

def paraboLa(a, b, c):
    """f, f', f'' and a plot label for y = ax^2 + bx + c."""
    terms = []
    for coef, power in ((a, "x^2"), (b, "x"), (c, "")):
        if coef:
            num = " " if abs(coef) == 1 and power else f"{abs(coef):g}"
            terms.append((" " if coef < 0 else "+ ") + num + power)
    label = " ".join(terms).replace("-", " - ", 1) if terms[0][0] == "-" else " "
    ".join(terms)[2:]
    return (lambda x: a * x**2 + b * x + c, lambda x: 2 * a * x + b, lambda x: 2 * a + 0 *
x,
        f"$y = {label}$")

def tag(x0, y0):
    """(-4, 0) -> 'm4_0' for file names."""
    fmt = lambda v: (f"m{-v:g}" if v < 0 else f"{v:g}").replace(".", "p")
    return f"{fmt(x0)}_{fmt(y0)}"

def solve_and_plot(name, f, df, ddf, eq, x0, y0, guess):
    lo, hi = choose_bracket(x0, y0, f)
    d_nr, x_nr, nr_hist = find_distance_newton(x0, y0, f, df, ddf, initial_guess=guess)
    d_gs, x_gs, gs_hist = golden_section_search(x0, y0, f, lo, hi)

```

```

print(f"{eq.strip('$')<22} ({x0:g}, {y0:g}) "
      f"Newton x*={x_nr:.6f} d={d_nr:.6f} ({len(nr_hist) - 1} it, start {guess:g}) "
      f"Golden x*={x_gs:.6f} d={d_gs:.6f} ({len(gs_hist) - 1} it, [{lo:g}, {hi:g}])")
plots.plot_newton(f, x0, y0, nr_hist, eq, os.path.join(MEDIA, f"nr_{name}{tag(x0,
y0)}.png"))
plots.plot_golden(f, x0, y0, gs_hist, eq, os.path.join(MEDIA, f"gss_{name}{tag(x0,
y0)}.png"))
return x_nr, nr_hist, gs_hist

# Other curves: (file prefix, f, f', f'', label, point, Newton start)
OTHER = [
    ("parab2_", *parabola(0.5, -2, 1), (5, -2), 2.0),
    ("exp_", np.exp, np.exp, np.exp, "$y = e^x$", (2, 1), 0.0),
    ("log_", np.log, lambda x: 1 / x, lambda x: -1 / x**2, r"$y = \ln x$", (0, 0), 1.0),
    ("rational_", lambda x: 4 / (1 + x**2), lambda x: -8 * x / (1 + x**2)**2,
     lambda x: (24 * x**2 - 8) / (1 + x**2)**3, r"$y = \dfrac{4}{1 + x^2}$", (3, 3), 2.0),
    ("sqrt_", np.sqrt, lambda x: 0.5 / np.sqrt(x), lambda x: -0.25 * x**-1.5, r"$y = \sqrt{x}$", (4, 0), 4.0),
]

if __name__ == "__main__":
    np.seterr(invalid="ignore", divide="ignore") # ln x and sqrt x are undefined for x ≤
0 on plot grids
    os.makedirs(MEDIA, exist_ok=True)
    plots.setup_style()

    if len(sys.argv) == 6:
        a, b, c, x0, y0 = map(float, sys.argv[1:])
        vertex = -b / (2 * a)
        solve_and_plot("custom_", *parabola(a, b, c), x0, y0, vertex + 1 if x0 == vertex
        else vertex)
        sys.exit()

    # Assignment: y = x^2 + 5. Newton starts at the vertex x = 0, except for (0, 0)
    # where that is already the answer, so it starts at x = 1 (same as the hand work).
    f, df, ddf, eq = parabola(1, 0, 5)
    points = [(0, 0), (-4, 0), (-8, 0), (2, 0), (6, 0)]
    x_stars, nr_all, gs_all = [], {}, {}
    for x0, y0 in points:
        x_nr, nr_hist, gs_hist = solve_and_plot("", f, df, ddf, eq, x0, y0, 1.0 if x0 == 0
        else 0.0)
        x_stars.append(x_nr)
        nr_all[f"({x0}, {y0})"] = nr_hist, gs_hist
    plots.plot_setup(f, points, eq, os.path.join(MEDIA, "p1_setup.png"))
    plots.plot_all_results(f, points, x_stars, eq, os.path.join(MEDIA,
    "p1_all_results.png"))
    plots.plot_convergence(nr_all, gs_all, os.path.join(MEDIA, "p1_convergence.png"))

    for name, f, df, ddf, eq, (x0, y0), guess in OTHER:
        solve_and_plot(name, f, df, ddf, eq, x0, y0, guess)

```

```
"""
Plotting helpers for SYDE 572 Assignment 1. These only draw: the solvers in
distance.py and fitting.py pass in the steps they recorded.
"""
```

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator

CURVE = "#2f5bd3"      # the function / the exact answer
POINT = "#1c1b19"      # query points, data points, final answer
MUTED = "#8a877f"      # guides and secondary text
ITER_CMAP = "Oranges"  # iterates: light (early) → dark (late)
```

```
def setup_style():
    plt.rcParams.update({
        "figure.facecolor": "white",
        "savefig.facecolor": "white",
        "font.size": 11,
        "axes.titlesize": 13,
        "axes.titleweight": "semibold",
        "axes.edgecolor": "#c9c6bd",
        "axes.linewidth": 0.8,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.grid": True,
        "grid.color": "#ebe9e3",
        "xtick.color": MUTED,
        "ytick.color": MUTED,
        "legend.frameon": False,
        "legend.fontsize": 10,
        "mathtext.fontset": "cm",
    })

def _iter_colors(n):
    cmap = plt.get_cmap(ITER_CMAP)
    return [cmap(v) for v in np.linspace(0.35, 0.95, max(n, 1))]
```

```
def _save(fig, fname, title):
    fig.suptitle(title, fontsize=15, fontweight="semibold", y=1.0)
    fig.tight_layout()
    fig.savefig(fname, dpi=200, bbox_inches="tight")
    plt.close(fig)
```

```
# =====
# Part 1: distance from a point to a curve
# =====
```

```

def _dist_sq(f, x0, y0):
    return lambda x: (x - x0) ** 2 + (f(x) - y0) ** 2

def _num_deriv(g, x, h=1e-5):
    """Central difference, used only to draw tangent lines."""
    return (g(x + h) - g(x - h)) / (2 * h)

def _result_box(ax, f, x0, y0, x_star):
    y_star = f(x_star)
    d = np.hypot(x_star - x0, y_star - y0)
    ax.text(0.5, -0.30, f"closest point ({x_star:.4f}, {y_star:.4f})    shortest distance d
= {d:.4f}",
            transform=ax.transAxes, ha="center", va="top", fontsize=10, family="monospace",
            bbox=dict(boxstyle="round,pad=0.5", fc="white", ec="#dcd9d0"))

def _curve_panel(ax, f, x0, y0, xs, eq, pad=1.5):
    """Curve and query point, framed around the point and the given x values."""
    xs = np.asarray(xs, float)
    lo, hi = min(x0, xs.min()) - pad, max(x0, xs.max()) + pad
    ys = np.append(f(xs), y0)
    ylo, yhi = ys.min() - pad, ys.max() + pad
    grid = np.linspace(lo, hi, 800)
    yg = f(grid)
    yg = np.where((yg > yhi + 5 * pad) | (yg < ylo - 5 * pad), np.nan, yg)
    ax.plot(grid, yg, color=CURVE, lw=2.2, label=eq, zorder=2)
    ax.scatter([x0], [y0], s=70, color=POINT, zorder=6, label=f"point $({x0:g}, \{y0:g\})$")
    ax.annotate(f"$({x0:g}, \{y0:g\})$", (x0, y0), xytext=(8, -16), textcoords="offset
points")
    ax.set_xlim(lo, hi)
    ax.set_ylim(ylo, yhi)
    ax.set_xlabel("$x$")
    ax.set_ylabel("$y$")
    ax.set_aspect("equal", adjustable="box")    # so the shortest segment looks
perpendicular

def plot_newton(f, x0, y0, nr_xs, eq, fname, max_labels=6):
    """Left: segment from the point to the curve at each iterate x_n.
    Right: D'(x) and the tangent used at each step; it crosses zero at the next iterate."""
    nr_xs = np.asarray(nr_xs, float)
    Dp = lambda x: _num_deriv(_dist_sq(f, x0, y0), x)
    cols = _iter_colors(len(nr_xs))
    fig, (axL, axR) = plt.subplots(1, 2, figsize=(13, 5.2), gridspec_kw={"width_ratios":
[1.1, 1]})

    _curve_panel(axL, f, x0, y0, nr_xs, eq)
    last = np.inf
    for n, (x, c) in enumerate(zip(nr_xs, cols)):
        final = n == len(nr_xs) - 1
        axL.plot([x0, x], [y0, f(x)], ls="--" if final else "--", lw=2.2 if final else 1.2,
                color=POINT if final else c, zorder=3)

```

```

        axL.scatter([x], [f(x)], s=46, color=c, edgecolor="white", lw=1.2, zorder=5)
        if n < max_labels and abs(x - last) > 0.08 * np.ptp(axL.get_xlim()):
            last = x
            axL.annotate(f"$x_{{{n}}}$", (x, f(x)), xytext=(6, 6), textcoords="offset
points", color="#6b3a12")
        axL.scatter([], [], color=cols[len(cols) // 2], label="iterates $x_n$ (light → dark)")
        axL.set_title(f"Iterates on the curve ({len(nr_xs) - 1} steps)")
        axL.legend(loc="upper center", bbox_to_anchor=(0.5, -0.12), ncol=3, fontsize=9)
        _result_box(axL, f, x0, y0, nr_xs[-1])

    span = max(np.ptp(nr_xs), 0.2)
    grid = np.linspace(nr_xs.min() - 0.35 * span, nr_xs.max() + 0.35 * span, 600)
    axR.plot(grid, Dp(grid), color=CURVE, lw=2.2, label="$D'(x)$")
    axR.axhline(0, color=MUTED, lw=1)
    last = np.inf
    for n in range(len(nr_xs) - 1):
        x, c = nr_xs[n], cols[n]
        seg = np.array([x, nr_xs[n + 1]])
        axR.plot(seg, Dp(x) + _num_deriv(Dp, x) * (seg - x), color=c, lw=1.4, ls="--")
        axR.plot([x, x], [0, Dp(x)], color=c, lw=0.9, ls=":")
        axR.scatter([x], [Dp(x)], s=40, color=c, edgecolor="white", lw=1, zorder=5)
        if n < max_labels and abs(x - last) > 0.06 * span and abs(x - nr_xs[-1]) > 0.06 *
span:
            last = x
            axR.annotate(f"$x_{{{n}}}$", (x, 0), xytext=(0, -16), textcoords="offset
points",
                        ha="center", color="#6b3a12")
    axR.scatter([nr_xs[-1]], [0], s=70, marker="*", color=POINT, zorder=6,
                label=f"root $x^* \approx \{nr_xs[-1]:.4f}$")
    axR.plot([], [], color=cols[0], ls="--", label="tangent at $x_n$")
    axR.set_xlabel("$x$")
    axR.set_ylabel("$D'(x)$")
    axR.set_title(r"Each step: follow the tangent of $D'$ to zero")
    axR.legend(loc="upper left")

    _save(fig, fname, f"Shortest distance from $({x0:g}, \{y0:g\})$ to {eq}")

def plot_golden(f, x0, y0, brackets, eq, fname, n_show=8):
    """Left: search interval on the curve. Middle: D(x) with the first n_show brackets
    stacked underneath. Right: bracket width per iteration (x0.618 each step)."""
    D = _dist_sq(f, x0, y0)
    a0, b0 = brackets[0]
    x_star = sum(brackets[-1]) / 2
    shown = brackets[:n_show]
    cols = _iter_colors(len(shown))
    fig, (axL, axM, axR) = plt.subplots(1, 3, figsize=(16, 5.2), gridspec_kw=
{"width_ratios": [1, 1.15, 0.85]})

    _curve_panel(axL, f, x0, y0, [a0, b0], eq, pad=1.0)
    axL.plot([x0, x_star], [y0, f(x_star)], color=POINT, lw=2.2, zorder=3)
    axL.scatter([x_star], [f(x_star)], s=60, color=POINT, marker="*", zorder=6)
    axL.axvspan(a0, b0, color=cols[0], alpha=0.18, lw=0, label=f"initial bracket [{a0:.3g},
{b0:.3g}]")
    axL.set_title("Search interval on the curve")

```

```

axL.legend(loc="upper center", bbox_to_anchor=(0.5, -0.12), ncol=2, fontsize=9)
_result_box(axL, f, x0, y0, x_star)

grid = np.linspace(a0, b0, 600)
Dg = D(grid)
axM.plot(grid, Dg, color=CURVE, lw=2.2)
h = np.ptp(Dg) or 1.0
step, base = 0.09 * h, Dg.min() - 0.12 * h
for n, ((a, b), c) in enumerate(zip(shown, cols)):
    yb = base - n * step
    axM.plot([a, b], [yb, yb], color=c, lw=5, solid_capstyle="round")
    axM.text(b + 0.01 * (b0 - a0), yb, f" {n}", va="center", fontsize=9,
color="#6b3a12")
    for x in (a, b):
        axM.plot([x, x], [yb, D(x)], color=c, lw=0.6, ls=":", alpha=0.7)
    axM.axvline(x_star, color=POINT, lw=1, ls="--")
    axM.annotate(f"$x^* \approx {x_star:.4f}$", (x_star, Dg.max()), xytext=(6, -4),
textcoords="offset points")
    axM.set_ylim(base - (len(shown) + 0.5) * step, Dg.max() + 0.08 * h)
    axM.set_xlabel("$x$")
    axM.set_ylabel("$D(x) = (x - x_0)^2 + (f(x) - y_0)^2$")
    axM.set_title(f"Bracket $[a, b]$ for the first {len(shown)} iterations")

widths = [b - a for a, b in brackets]
axR.semilogy(widths, color=CURVE, lw=2, marker="o", ms=4)
axR.set_xlabel("iteration")
axR.set_ylabel("bracket width $b - a$")
axR.set_title(f"Bracket width ({len(widths)} - 1) steps, ×0.618 each")

_save(fig, fname, f"Golden section search: $({x0:g}, \{y0:g})$ to {eq}")

def plot_setup(f, points, eq, fname):
    """The curve and all query points, before solving anything."""
    xs, ys = zip(*points)
    lo, hi = min(xs) - 1.5, max(xs) + 1.5
    ylo = min(ys) - 2
    yhi = ylo + 0.6 * (hi - lo)
    grid = np.linspace(lo, hi, 800)
    fig, ax = plt.subplots(figsize=(9, 5.5))
    ax.plot(grid, f(grid), color=CURVE, lw=2.2, label=eq)
    ax.scatter(xs, ys, s=60, color=POINT, zorder=5, label="query points")
    for px, py in points:
        ax.annotate(f"$({px:g}, \{py:g})$", (px, py), xytext=(0, 10), textcoords="offset
points", ha="center")
    ax.set_xlim(lo, hi)
    ax.set_ylim(ylo, yhi)
    ax.set_xlabel("$x$")
    ax.set_ylabel("$y$")
    ax.set_title("Problem setup")
    ax.legend(loc="upper left")
    fig.tight_layout()
    fig.savefig(fname, dpi=200, bbox_inches="tight")
    plt.close(fig)

```

```

def plot_all_results(f, points, x_stars, eq, fname):
    """Every point joined to its closest point on the curve."""
    x_stars = np.asarray(x_stars, float)
    xs = [p[0] for p in points] + list(x_stars)
    lo, hi = min(xs) - 1.5, max(xs) + 1.5
    ylo = min(min(p[1] for p in points), f(x_stars).min()) - 2
    yhi = max(max(p[1] for p in points), f(x_stars).max()) + 3
    grid = np.linspace(lo, hi, 800)
    yg = f(grid)
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.plot(grid, np.where(yg > yhi + 10, np.nan, yg), color=CURVE, lw=2.2, label=eq,
            zorder=2)
    for (px, py), xs_ in zip(points, x_stars):
        ys_ = f(xs_)
        ax.plot([px, xs_], [py, ys_], color=POINT, lw=1.4, ls="--", zorder=3)
        ax.scatter([px], [py], s=55, color=POINT, zorder=5)
        ax.scatter([xs_], [ys_], s=55, color="white", edgecolor=POINT, lw=1.6, zorder=6)
        ax.annotate(f"${px:g},\\,\\,${py:g})$\\n$d={np.hypot(xs_ - px, ys_ - py):.3f}$", (px,
        py),
                    xytext=(0, -14), textcoords="offset points", ha="center", va="top",
                    fontsize=10)
        ax.scatter([], [], color=POINT, label="query point")
        ax.scatter([], [], color="white", edgecolor=POINT, label="closest point on curve")
        ax.set_xlim(lo, hi)
        ax.set_ylim(ylo - 1.5, yhi)
        ax.set_aspect("equal", adjustable="box")
        ax.set_xlabel("$x$")
        ax.set_ylabel("$y$")
        ax.set_title("Shortest distance from each point to the curve")
        ax.legend(loc="lower center", ncol=3)
        fig.tight_layout()
        fig.savefig(fname, dpi=200, bbox_inches="tight")
        plt.close(fig)

def plot_convergence(nr_histories, gss_histories, fname):
    """Newton:  $|x_n - x^*|$ . Golden section: the error bound  $(b - a)/2$ ."""
    fig, (axA, axB) = plt.subplots(1, 2, figsize=(13, 4.6), sharey=True)
    cmap = plt.get_cmap("Blues")
    n = max(len(nr_histories) - 1, 1)
    for i, (label, xs) in enumerate(nr_histories.items()):
        xs = np.asarray(xs, float)
        axA.semilogy(np.maximum(np.abs(xs - xs[-1])[:-1], 1e-16), marker="o", ms=5, lw=1.8,
                     color=cmap(0.45 + 0.5 * i / n), label=label)
    for i, (label, br) in enumerate(gss_histories.items()):
        axB.semilogy([(b - a) / 2 for a, b in br], lw=1.8, color=cmap(0.45 + 0.5 * i / n),
                     label=label)
    axA.set_title("Newton-Raphson:  $|x_n - x^*|$ ")
    axB.set_title("Golden section: error bound  $(b-a)/2$ ")
    axA.xaxis.set_major_locator(MaxNLocator(integer=True))
    for ax in (axA, axB):
        ax.set_xlabel("iteration")
        ax.legend(title="point", fontsize=9)
    axA.set_ylabel("error in  $x$ ")

```

```

_save(fig, fname, "Convergence speed")

# =====
# Part 2: least-squares fitting
# =====
def _poly_label(c):
    terms = [f"{c[0]:.3f}"] + [f"{abs(v):.3f}" + ("x" if k == 1 else f"x^{k}") for k, v in
enumerate(c) if k]
    signs = [""] + [" - " if v < 0 else " + " for v in c[1:]]
    return "$y = " + "".join(s + t for s, t in zip(signs, terms)) + "$"

def _poly(c, x):
    return sum(ck * x ** k for k, ck in enumerate(c))

def plot_fit_steps(name, X, Y, mse, coord_path, newton_path, c_exact, fname, show=(0, 1,
2)):
    """
    coord_path : parameters after every single-parameter update (one-at-a-time Newton)
    newton_path : parameters after every full Newton step
    Left : data with the fit after iterations `show` and the final fit.
    Middle: line → path over the MSE contours (c0, c1); parabola → each parameter vs
iteration.
    Right : MSE - MSE* per iteration for both Newton variants.
    """
    p = len(c_exact)
    iters = coord_path[:, :p]
    if len(coord_path) % p != 1:
        iters = np.vstack([iters, coord_path[-1]])
    mse_star = mse(c_exact)
    fig, (axL, axM, axR) = plt.subplots(1, 3, figsize=(17, 5.4), gridspec_kw=
{"width_ratios": [1.1, 1, 1]})

    # ---- left: fits over the data ----
    xg = np.linspace(X.min() - 0.3, X.max() + 0.3, 300)
    cols = _iter_colors(len(show))
    for k, col in zip(show, cols):
        axL.plot(xg, _poly(iters[k], xg), color=col, lw=1.6, ls="--",
label=f"iter {k}: MSE = {mse(iters[k]):.4f}")
        axL.plot(xg, _poly(c_exact, xg), color=CURVE, lw=2.4,
label=f"final ({len(iters) - 1} iters): MSE = {mse(iters[-1]):.4f}")
    axL.scatter(X, Y, s=60, color=POINT, zorder=5, label="data")
    axL.set_xlabel("$x$")
    axL.set_ylabel("$y$")
    axL.set_title("Fit after each iteration")
    axL.legend(loc="upper left", fontsize=9)
    axL.text(0.98, 0.03, _poly_label(c_exact), transform=axL.transAxes, ha="right",
fontsize=11, color=CURVE)

    # ---- middle ----
    if p == 2:
        pts = np.vstack([coord_path, newton_path, c_exact])
        span = np.ptp(pts, axis=0) + 0.6

```

```

g0 = np.linspace(pts[:, 0].min() - 0.2 * span[0], pts[:, 0].max() + 0.2 * span[0],
200)
g1 = np.linspace(pts[:, 1].min() - 0.2 * span[1], pts[:, 1].max() + 0.2 * span[1],
200)
C0, C1 = np.meshgrid(g0, g1)
Z = np.vectorize(lambda a, b: mse(np.array([a, b])))(C0, C1)
axM.contour(C0, C1, Z, levels=np.geomspace(mse_star * 1.01, Z.max(), 14),
colors=MUTED, linewidths=0.6)
axM.plot(coord_path[:, 0], coord_path[:, 1], color=_iter_colors(3)[1], lw=1.6,
marker="o", ms=3.5, label="one parameter at a time")
for k in show[1:]:
    axM.annotate(f"iter {k}", iters[k], xytext=(8, -4), textcoords="offset points",
fontsize=9, color="#6b3a12")
    axM.plot(newton_path[:, 0], newton_path[:, 1], color=CURVE, lw=2, ls="--",
marker="s", ms=5,
label="full Newton (1 step)")
    axM.scatter(*c_exact, s=160, marker="*", color=POINT, zorder=6, label=f"optimum
({c_exact[0]:.2f}, {c_exact[1]:.2f})")
    axM.annotate("start", coord_path[0], xytext=(6, 6), textcoords="offset points",
fontsize=9)
    axM.set_xlabel("$c_0$ (intercept)")
    axM.set_ylabel("$c_1$ (slope)")
    axM.set_title("Path over the MSE contours")
    axM.legend(loc="lower left", fontsize=9)
else:
    its = np.arange(len(iters))
    for j, col in enumerate(("c2562f", "#2e7d4f", CURVE)):
        axM.plot(iters + 1, iters[:, j], color=col, lw=2, label=f"$c_{j+1}$")
        axM.axhline(c_exact[j], color=col, lw=1, ls=":")
        axM.text(len(iters) * 1.05, c_exact[j], f"{c_exact[j]:.2f}", color=col,
va="center", fontsize=10)
    axM.set_xscale("log")
    axM.set_xlabel("iteration + 1 (log scale)")
    axM.set_ylabel("parameter value")
    axM.set_title("Parameters, one at a time (dotted = exact)")
    axM.legend(loc="upper right")

# ---- right: convergence ----
excess = lambda cs: np.maximum([mse(c) - mse_star for c in cs], 1e-16)
axR.semilogy(excess(iters), color=_iter_colors(3)[1], lw=2, label=f"one at a time
({len(iters) - 1} iters)")
axR.semilogy(excess(newton_path), color=CURVE, lw=2, ls="--", marker="s", ms=5,
label="full Newton (1 step)")
axR.set_xlabel("iteration")
axR.set_ylabel("MSE - MSE*")
axR.set_title(f"Convergence to MSE* = {mse_star:.4f}")
axR.legend(loc="upper right")

_save(fig, fname, f"Fitting a {name}: Newton-Raphson intermediate steps")

def plot_final_fits(X, Y, fits, fname):
    """fits: list of (name, c, mse)."""
    xg = np.linspace(X.min() - 0.3, X.max() + 0.3, 300)
    fig, ax = plt.subplots(figsize=(9, 5.5))

```

```

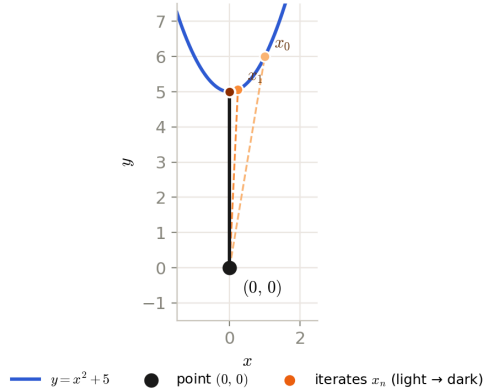
for (name, c, m), col in zip(fits, ("c2562f", CURVE)):
    ax.plot(xg, _poly(c, xg), color=col, lw=2.2, label=f"{name}: {_poly_label(c)}, MSE
= {m:.4f}")
    for x, y in zip(X, Y):
        ax.plot([x, x], [y, _poly(c, x)], color=col, lw=1, ls=":")
    ax.scatter(X, Y, s=60, color=POINT, zorder=5, label="data")
    ax.set_xlabel("$x$")
    ax.set_ylabel("$y$")
    ax.set_title("Least-squares line vs parabola (dotted = residuals)")
    ax.legend(loc="upper left")
    fig.tight_layout()
    fig.savefig(fname, dpi=200, bbox_inches="tight")
    plt.close(fig)

```

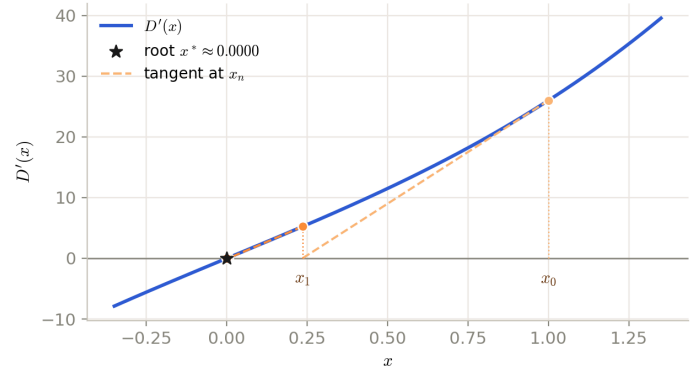
1.6 Newton–Raphson: intermediate steps

Shortest distance from $(0, 0)$ to $y = x^2 + 5$

Iterates on the curve (4 steps)



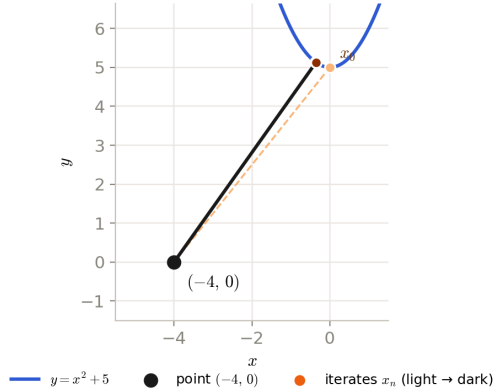
Each step: follow the tangent of D' to zero



$(0, 0)$. Newton–Raphson iterates.

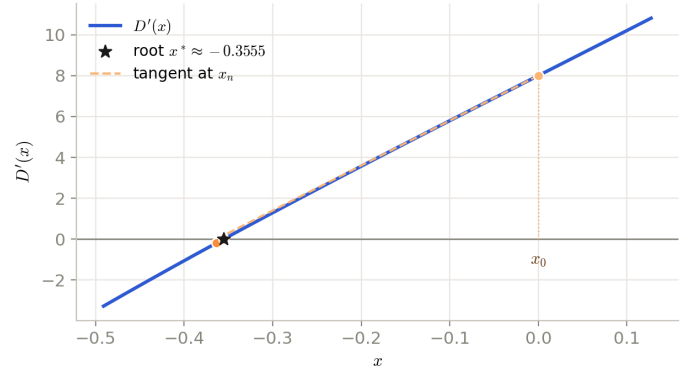
Shortest distance from $(-4, 0)$ to $y = x^2 + 5$

Iterates on the curve (4 steps)



closest point $(-0.3555, 5.1264)$ shortest distance $d = 6.2898$

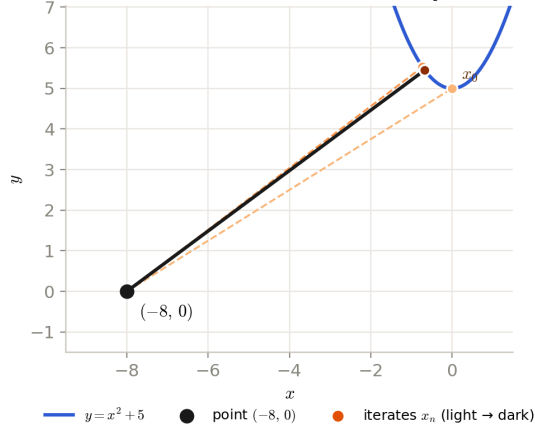
Each step: follow the tangent of D' to zero



$(-4, 0)$. Newton-Raphson iterates.

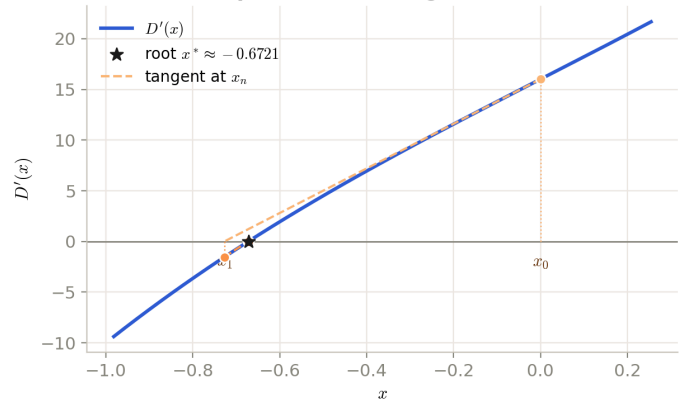
Shortest distance from $(-8, 0)$ to $y = x^2 + 5$

Iterates on the curve (5 steps)



closest point $(-0.6721, 5.4517)$ shortest distance $d = 9.1334$

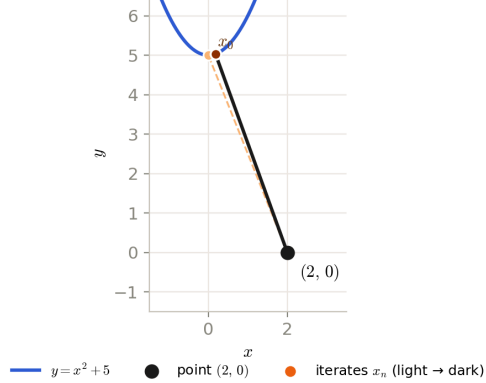
Each step: follow the tangent of D' to zero



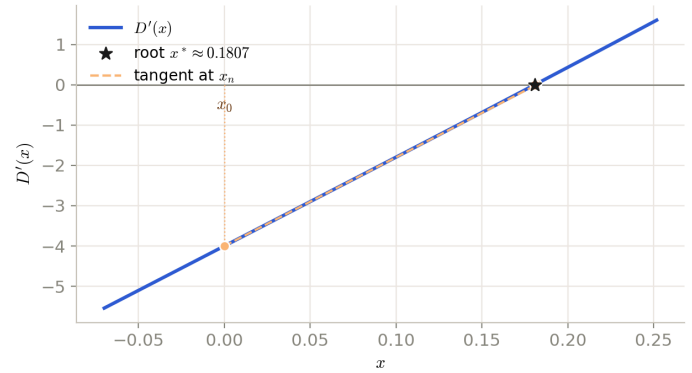
$(-8, 0)$. Newton-Raphson iterates.

Shortest distance from $(2, 0)$ to $y = x^2 + 5$

Iterates on the curve (4 steps)



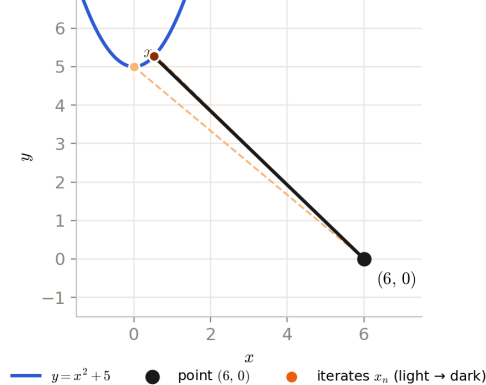
Each step: follow the tangent of D' to zero



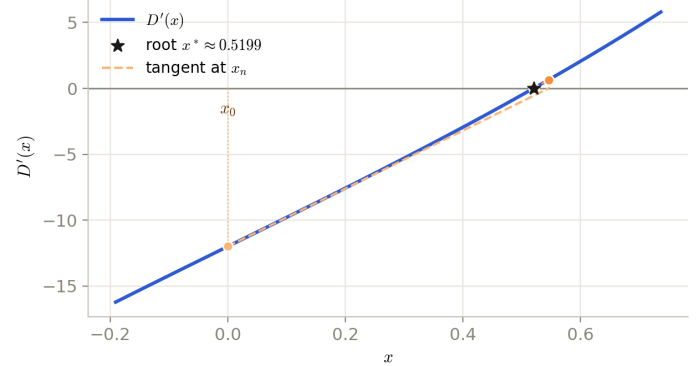
$(2, 0)$. Newton–Raphson iterates.

Shortest distance from $(6, 0)$ to $y = x^2 + 5$

Iterates on the curve (4 steps)



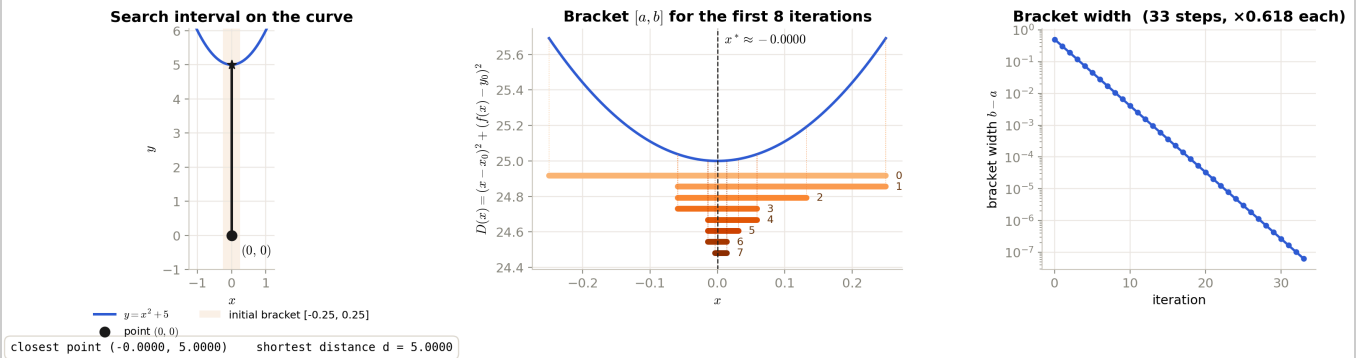
Each step: follow the tangent of D' to zero



$(6, 0)$. Newton–Raphson iterates.

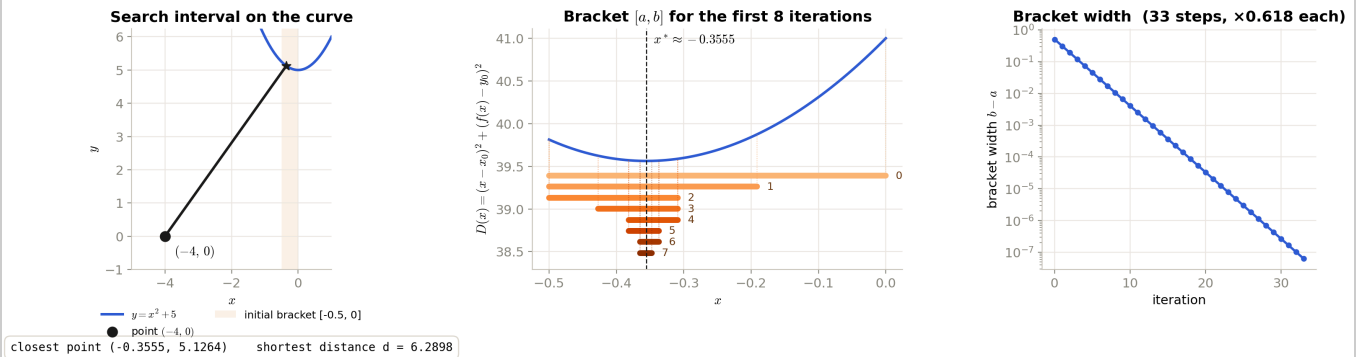
1.7 Golden section search: intermediate steps

Golden section search: $(0, 0)$ to $y = x^2 + 5$



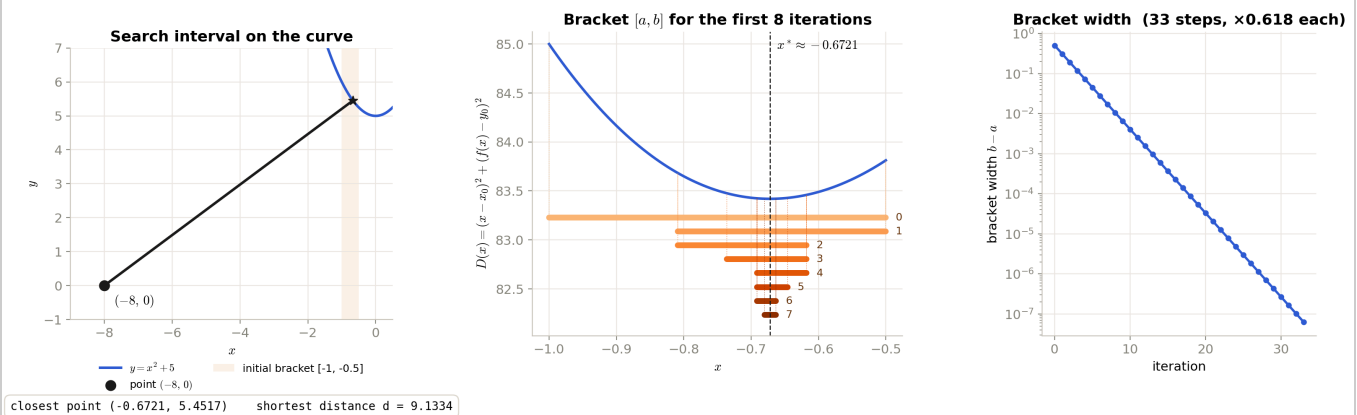
$(0, 0)$. Shrinking brackets.

Golden section search: $(-4, 0)$ to $y = x^2 + 5$



$(-4, 0)$. Shrinking brackets.

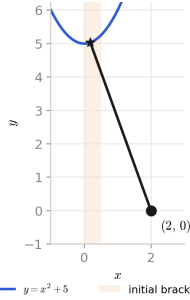
Golden section search: $(-8, 0)$ to $y = x^2 + 5$



$(-8, 0)$. Shrinking brackets.

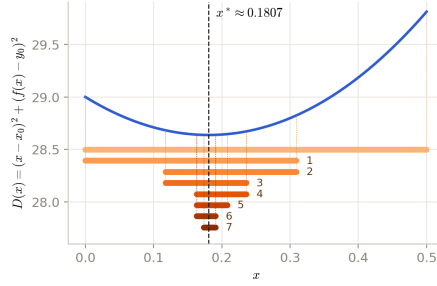
Golden section search: (2, 0) to $y = x^2 + 5$

Search interval on the curve

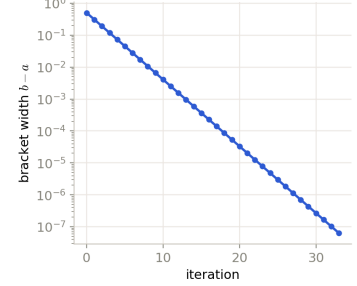


closest point (0.1807, 5.0327) shortest distance $d = 5.3514$

Bracket $[a, b]$ for the first 8 iterations



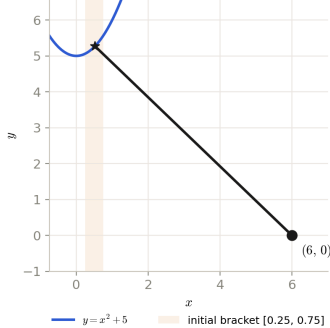
Bracket width (33 steps, $\times 0.618$ each)



(2, 0). Shrinking brackets.

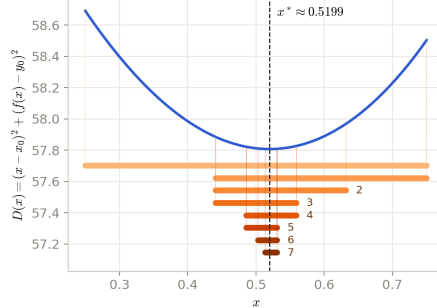
Golden section search: (6, 0) to $y = x^2 + 5$

Search interval on the curve

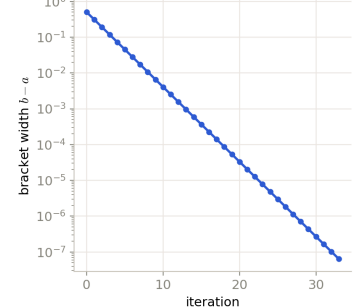


closest point (0.5199, 5.2703) shortest distance $d = 7.6031$

Bracket $[a, b]$ for the first 8 iterations



Bracket width (33 steps, $\times 0.618$ each)



(6, 0). Shrinking brackets.

1.8 Results & comparison

Newton–Raphson starts at the vertex (or 1 unit away when the point is directly below it); golden section uses a bracket around the lowest sample of $D(x)$. Both stop at a tolerance of 10^{-7} .

Point	Analytical		Newton–Raphson			Golden section		
	x^*	d	x^*	d	iters	x^*	d	iters
(0, 0)	0.000000	5.000000	0.000000	5.000000	4	0.000000	5.000000	33
(-4, 0)	-0.355470	6.289845	-0.355470	6.289845	4	-0.355470	6.289845	33
(-8, 0)	-0.672078	9.133420	-0.672078	9.133420	5	-0.672078	9.133420	33
(2, 0)	0.180745	5.351396	0.180745	5.351396	4	0.180745	5.351396	33
(6, 0)	0.519904	7.603125	0.519904	7.603125	4	0.519904	7.603125	33

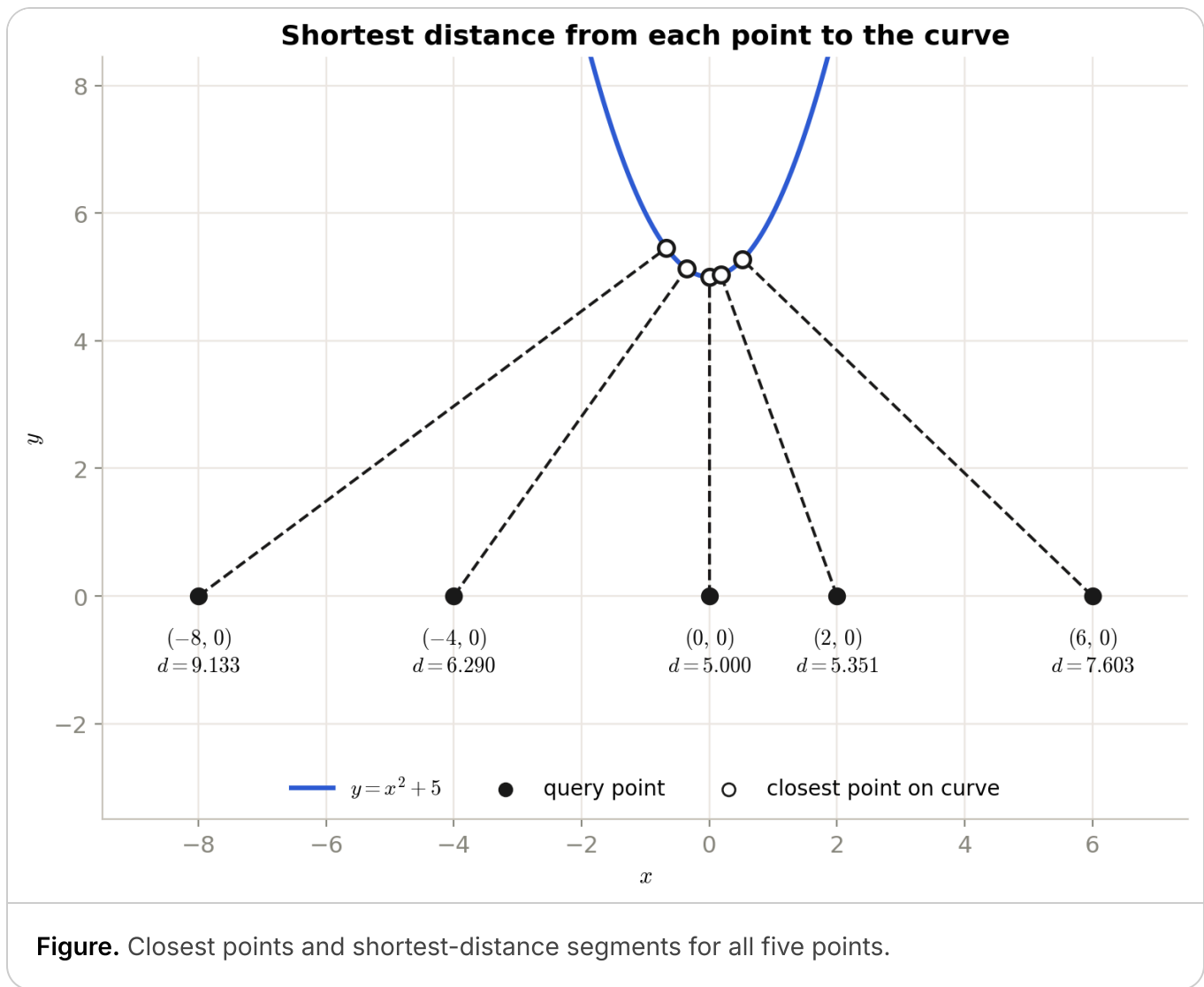


Figure. Closest points and shortest-distance segments for all five points.

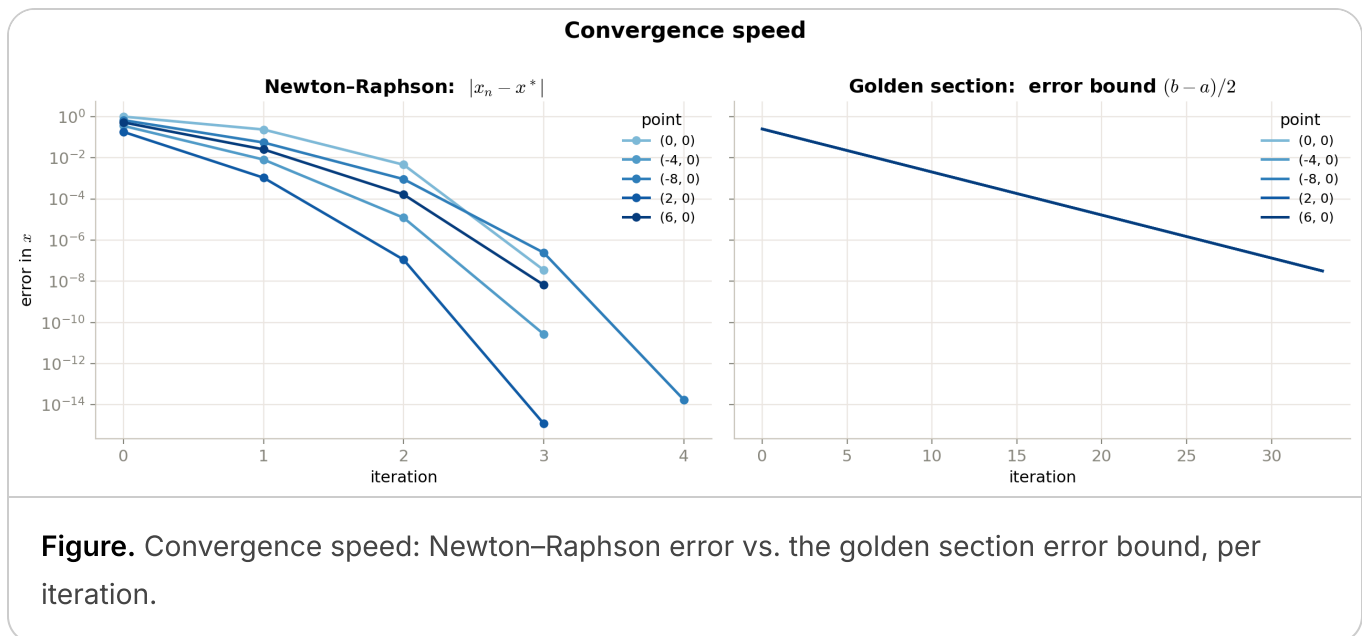


Figure. Convergence speed: Newton-Raphson error vs. the golden section error bound, per iteration.

Discussion

All three methods agree to six decimal places. The analytical method is exact, but only because D' is a cubic with a closed-form root. Newton–Raphson needs D' and D'' and a reasonable starting guess, but it converges quadratically: 4–5 iterations, with the number of correct digits roughly doubling each step. Golden section needs only $D(x)$ and a bracket containing one minimum. It always converges, but only linearly ($\times 0.618$ per step), so reaching 10^{-7} takes 33 iterations. Use Newton when derivatives are available and a good start is known; use golden section when they aren't, or when robustness matters more than speed.

1.9 Other functions & points

Nothing in the solvers is specific to parabolas. They only call f (and f' , f'' for Newton), so supporting another curve just means passing different functions. The one change is in the plotting grids, where NaNs from $\ln x$ and $\sqrt{x}$ at $x \leq 0$ are ignored. The golden section bracket is found the same way as before, by sampling $D(x)$ on $[x_0 - 10, x_0 + 10]$.

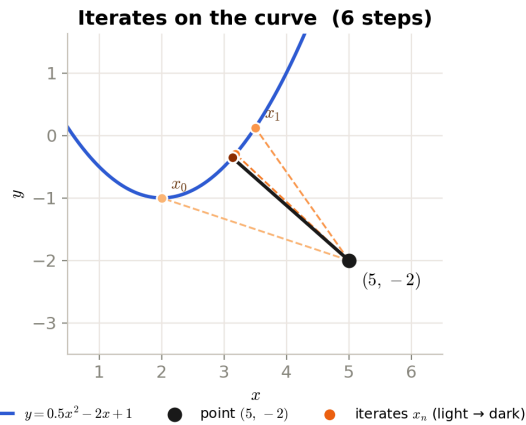
Function	Point	Newton–Raphson			Golden section		
		x^*	d	iters	x^*	d	iters
$y = 0.5x^2 - 2x + 1$	(5, -2)	3.134728	2.486228	6	3.134728	2.486228	33
$y = e^x$ (exponential)	(2, 1)	0.582871	1.623025	6	0.582871	1.623025	33
$y = \ln x$ (logarithmic)	(0, 0)	0.652919	0.779767	6	0.652919	0.779767	33
$y = \frac{4}{1+x^2}$ (rational)	(3, 3)	1.000000	2.236068	7	1.000000	2.236068	33
$y = \sqrt{x}$ (radical)	(4, 0)	3.500000	1.936492	2	3.500000	1.936492	33

Two checks by hand. For $\sqrt{x}$ from (4, 0), $D'(x) = 2(x - 4) + 1 = 0$ gives exactly $x = 3.5$, and D' is linear, so Newton needs only one real step. For $\frac{4}{1+x^2}$ from (3, 3), the answer (1, 2) gives $d = \sqrt{5}$, and the tangent slope there, -2 , is perpendicular to the segment (2, 1).

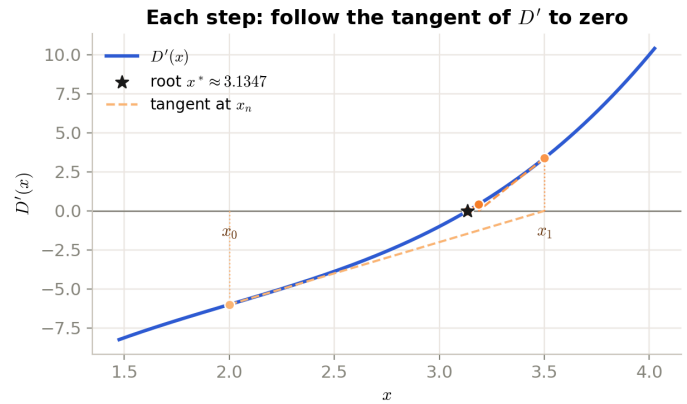
Newton needs a good start

On $y = \frac{4}{1+x^2}$, Newton started from $x = 3$ or $x = 1.5$ does not converge within 100 iterations: the tangent steps on D' overshoot back and forth. From $x = 2$ it converges in 7. Golden section has no starting guess to get wrong; it only needs a bracket.

Shortest distance from $(5, -2)$ to $y = 0.5x^2 - 2x + 1$

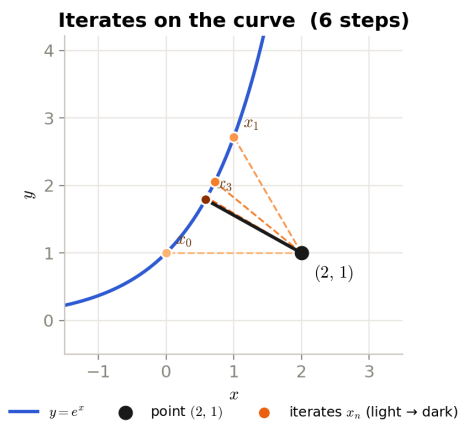


closest point $(3.1347, -0.3562)$ shortest distance $d = 2.4862$

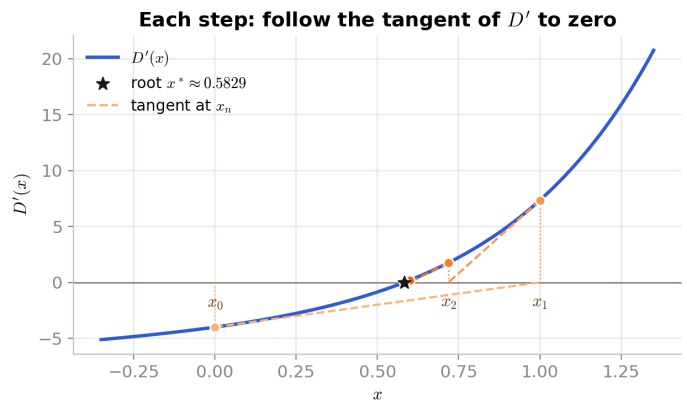


Another parabola. $y = 0.5x^2 - 2x + 1$ from $(5, -2)$.

Shortest distance from $(2, 1)$ to $y = e^x$

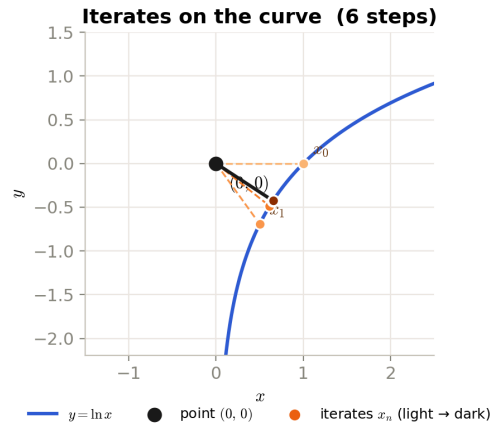


closest point $(0.5829, 1.7912)$ shortest distance $d = 1.6230$

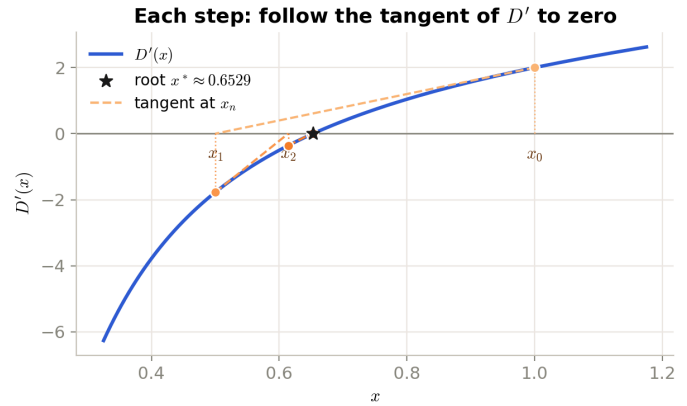


Exponential. $y = e^x$ from $(2, 1)$.

Shortest distance from $(0, 0)$ to $y = \ln x$

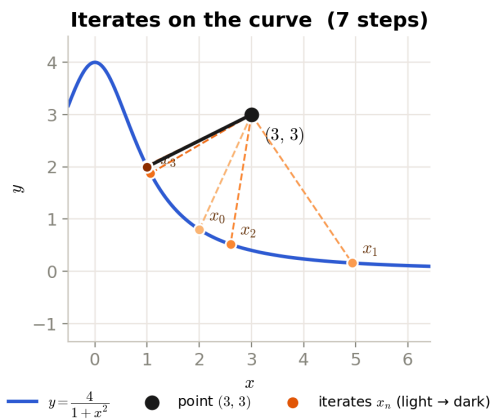


closest point $(0.6529, -0.4263)$ shortest distance $d = 0.7798$

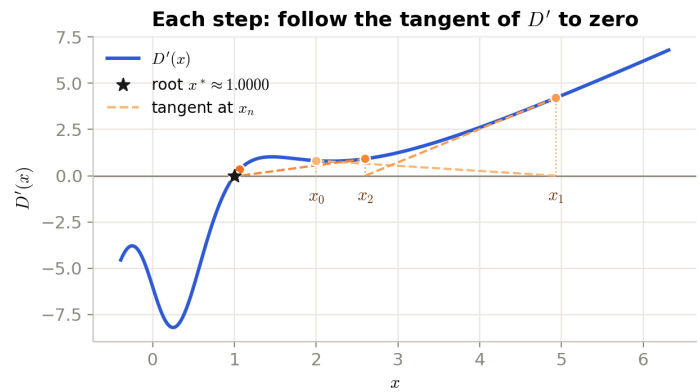


Logarithmic. $y = \ln x$ from $(0, 0)$.

Shortest distance from $(3, 3)$ to $y = \frac{4}{1+x^2}$



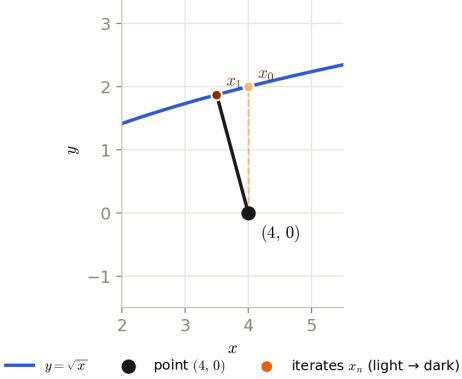
closest point $(1.0000, 2.0000)$ shortest distance $d = 2.2361$



Rational. $y = \frac{4}{1+x^2}$ from $(3, 3)$, starting at $x = 2$.

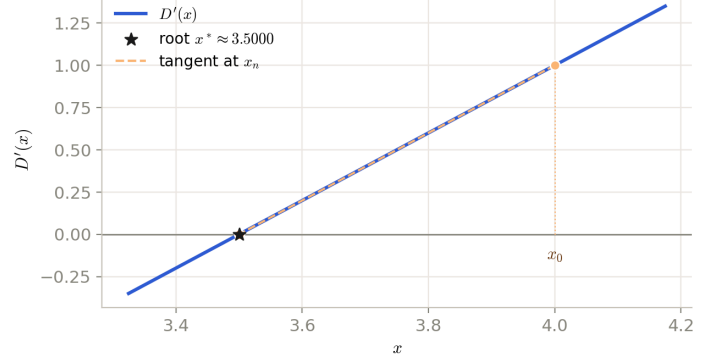
Shortest distance from $(4, 0)$ to $y = \sqrt{x}$

Iterates on the curve (2 steps)



closest point $(3.5000, 1.8708)$ shortest distance $d = 1.9365$

Each step: follow the tangent of D' to zero

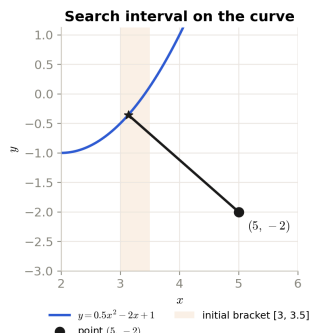


Radical. $y = \sqrt{x}$ from $(4, 0)$.

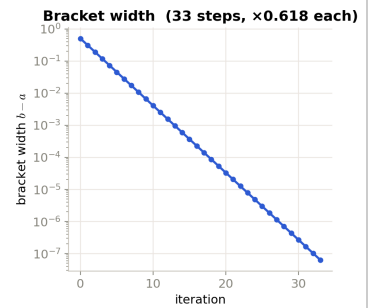
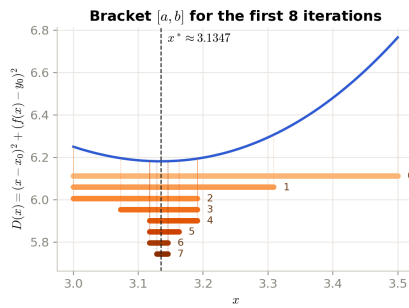
Golden section plots for these functions

click to expand

Golden section search: $(5, -2)$ to $y = 0.5x^2 - 2x + 1$

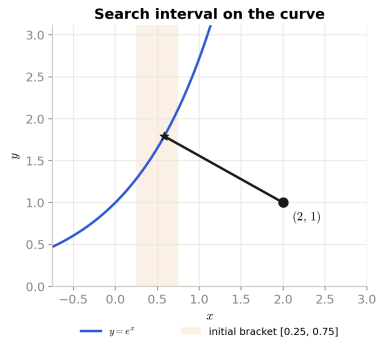


closest point $(3.1347, -0.3562)$ shortest distance $d = 2.4862$

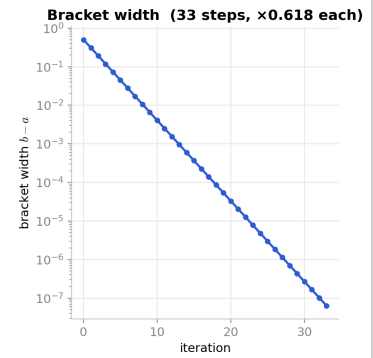
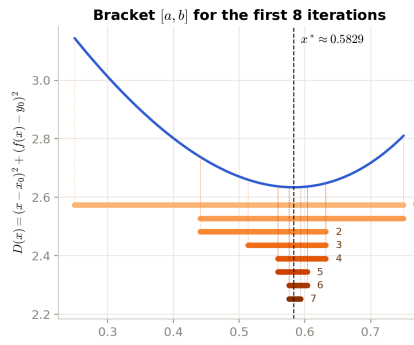


Another parabola. $y = 0.5x^2 - 2x + 1$ from $(5, -2)$.

Golden section search: (2, 1) to $y = e^x$

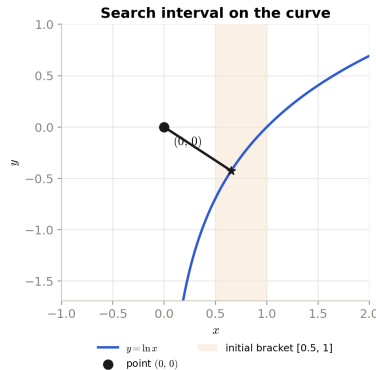


closest point (0.5829, 1.7912) shortest distance $d = 1.6230$

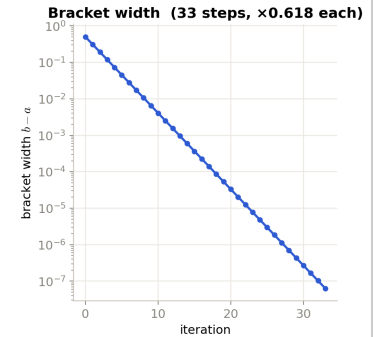
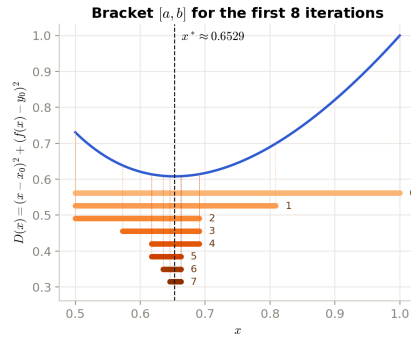


Exponential. $y = e^x$ from (2, 1).

Golden section search: (0, 0) to $y = \ln x$

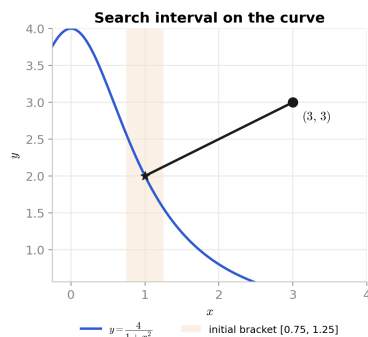


closest point (0.6529, -0.4263) shortest distance $d = 0.7798$

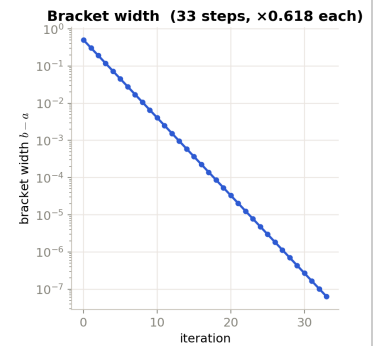
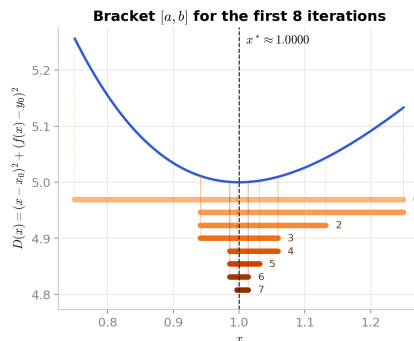


Logarithmic. $y = \ln x$ from (0, 0).

Golden section search: (3, 3) to $y = \frac{4}{1+x^2}$



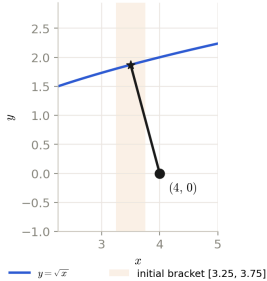
closest point (1.0000, 2.0000) shortest distance $d = 2.2361$



Rational. $y = \frac{4}{1+x^2}$ from (3, 3).

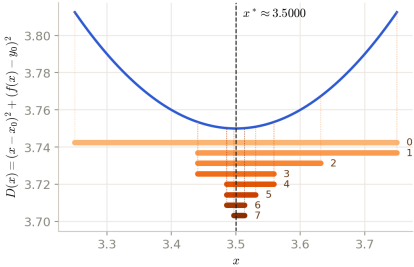
Golden section search: $(4, 0)$ to $y = \sqrt{x}$

Search interval on the curve

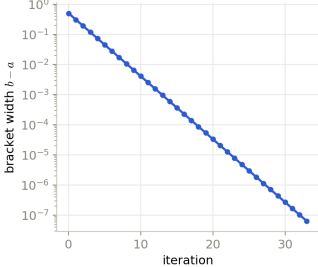


closest point $(3.5000, 1.8708)$ shortest distance $d = 1.9365$

Bracket $[a, b]$ for the first 8 iterations



Bracket width (33 steps, $\times 0.618$ each)



Radical. $y = \sqrt{x}$ from $(4, 0)$.

Fitting a line and a parabola

2.1 Data & the MSE objective

Data points: (0, 0.5), (1, 1.5), (2, 3.5), (3, 7.5). With error $e(x_i) = \hat{y}(x_i) - y_i$:

$$\begin{aligned} \text{line: } \hat{y} &= c_0 + c_1 x & \text{parabola: } \hat{y} &= c_0 + c_1 x + c_2 x^2 \\ \rho(c) &= \sum_{i=1}^4 e(x_i)^2, & \text{MSE} &= \frac{\rho}{N} = \frac{\rho}{4} \end{aligned}$$

The hand work minimizes the squared error ρ ; dividing by $N = 4$ gives the MSE and does not move the minimum. ρ is quadratic in the parameters, so each partial derivative is linear and setting them to zero gives a linear system (the normal equations). Some gradient equations are written divided by 2 (e.g. $\partial\rho/\partial c_0 = 2(4c_0 + 6c_1 - 13)$). That doesn't change their roots or the Newton steps, which only use the ratio of first to second derivative.

2.2 Analytical solution

Line

$$\frac{\partial\rho}{\partial c_0} = 0 : 4c_0 + 6c_1 = 13 \qquad \frac{\partial\rho}{\partial c_1} = 0 : 12c_0 + 28c_1 = 62$$

Subtracting the second equation from 3 × the first gives $-10c_1 + 23 = 0$, so $c_1 = 2.3$ and then $c_0 = -0.2$. The best line is $\hat{y} = -0.2 + 2.3x$ with $\rho = 2.3$, so **MSE = 0.575**.

Parabola

$$\begin{bmatrix} 4 & 6 & 14 \\ 6 & 14 & 36 \\ 14 & 36 & 98 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 13 \\ 31 \\ 83 \end{bmatrix} \qquad (A^\top A c = A^\top y)$$

I solved the three normal equations by Gauss–Jordan elimination on the augmented matrix, getting $c = (0.55, 0.05, 0.75)$. The best parabola is $\hat{y} = 0.55 + 0.05x + 0.75x^2$ with $\rho = 0.05$, so **MSE = 0.0125**.

PART TWO: FITTING TO AN EQUATION

ANALYTICAL METHOD

minimizing for line:

determine c_0, c_1 to minimize $p(c_0, c_1)$ where $e(x) = c_0 + c_1 x - y(x)$

For $(0, 0.5), (2, 3.5), (1, 1.5), (3, 7.5)$:

$$e(0) = c_0 - 0.5$$

$$e(1) = c_0 + c_1 - 1.5$$

$$e(2) = c_0 + 2c_1 - 3.5$$

$$e(3) = c_0 + 3c_1 - 7.5$$

So:

this is squared error.

$$p(c_0, c_1) = (c_0 - 0.5)^2 + (c_0 + c_1 - 1.5)^2 + (c_0 + 2c_1 - 3.5)^2 + (c_0 + 3c_1 - 7.5)^2$$

where it's minimum if $\frac{\partial p}{\partial c_0} = \frac{\partial p}{\partial c_1} = 0$

$$\begin{aligned}\frac{\partial p}{\partial c_0} &= 2(c_0 - 0.5)(1) + 2(c_0 + c_1 - 1.5)(1) + 2(c_0 + 2c_1 - 3.5)(1) + 2(c_0 + 3c_1 - 7.5)(1) \\ &= 2c_0 - 1 + 2c_0 + 2c_1 - 3 + 2c_0 + 4c_1 - 7 + 2c_0 + 6c_1 - 15\end{aligned}$$

$$\therefore \frac{\partial p}{\partial c_0} = 8c_0 + 12c_1 - 26 = 4c_0 + 6c_1 - 13 = 0$$

$$\begin{aligned}\frac{\partial p}{\partial c_1} &= 2(c_0 - 0.5)(0) + 2(c_0 + c_1 - 1.5)(1) + 2(c_0 + 2c_1 - 3.5)(2) + 2(c_0 + 3c_1 - 7.5)(3) \\ &= 2c_0 + 2c_1 - 3 + 4c_0 + 8c_1 - 14 + 6c_0 + 18c_1 - 45\end{aligned}$$

$$\therefore \frac{\partial p}{\partial c_1} = 12c_0 + 28c_1 - 62 = 0$$

$$\textcircled{1} \quad 4c_0 + 6c_1 - 13 = 0$$

$$\textcircled{2} \quad 12c_0 + 28c_1 - 62 = 0$$

solve. multiply $\textcircled{1}$ through by 3:

$$\textcircled{1} \quad 12c_0 + 18c_1 - 39 = 0$$

$$- \textcircled{2} \quad 12c_0 + 28c_1 - 62 = 0$$

$$-10c_1 + 23 = 0$$

$$\Rightarrow c_1 = 2.3$$

$$\text{then } 4c_0 + 6(2.3) - 13 = 0$$

$$\Rightarrow c_0 = -0.2$$

$\therefore$ The best (least squares) approximation: $y^*(x) = -0.2 + 2.3x$

To find MSE, plug c_0 & c_1 back into $p(c_0, c_1)$ for squared error then divide by N for mean

$$\begin{aligned}p(-0.2, 2.3) &= (-0.2 - 0.5)^2 + (-0.2 + 2.3 - 1.5)^2 + (-0.2 + 2(2.3) - 3.5)^2 + (-0.2 + 3(2.3) - 7.5)^2 \\ &= 0.85 + 1.45 \\ &= 2.3\end{aligned}$$

$$MSE = p/N = 2.3/4 = 0.575$$

$$\therefore \text{MSE for line is } 0.575$$

Hand work. Analytical solution: line.

minimizing for parabola:

$$y^*(x) = c_0 + c_1x + c_2x^2 \quad \text{where } e(x) = y^*(x) - y(x) = c_0 + c_1x + c_2x^2 - y(x)$$

$$e(0) = c_0 - 0.5$$

$$e(1) = c_0 + c_1 + c_2 - 1.5$$

$$e(2) = c_0 + 2c_1 + 4c_2 - 3.5$$

$$e(3) = c_0 + 3c_1 + 9c_2 - 7.5$$

$$\star \rho(c_0, c_1, c_2) = (c_0 - 0.5)^2 + (c_0 + c_1 + c_2 - 1.5)^2 + (c_0 + 2c_1 + 4c_2 - 3.5)^2 + (c_0 + 3c_1 + 9c_2 - 7.5)^2$$

$$\frac{\partial \rho}{\partial c_0} = 2[4c_0 + 6c_1 + 14c_2 - 13]$$

$$= 8c_0 + 12c_1 + 28c_2 - 26 \Rightarrow 4c_0 + 6c_1 + 14c_2 = 13$$

$$\frac{\partial \rho}{\partial c_1} = (2c_0 + 2c_1 + 2c_2 - 3) + (4c_0 + 8c_1 + 16c_2 - 14) + (6c_0 + 18c_1 + 54c_2 - 45)$$

$$= 12c_0 + 28c_1 + 72c_2 - 62 \Rightarrow 6c_0 + 14c_1 + 36c_2 = 31$$

$$\frac{\partial \rho}{\partial c_2} = (2c_0 + 2c_1 + 2c_2 - 3) + (8c_0 + 16c_1 + 32c_2 - 28) + (18c_0 + 54c_1 + 182c_2 - 135)$$

$$= 28c_0 + 72c_1 + 196c_2 - 166 \Rightarrow 14c_0 + 36c_1 + 98c_2 = 83$$

$$A^T A = \begin{bmatrix} 4 & 6 & 14 \\ 6 & 14 & 36 \\ 14 & 36 & 98 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 13 \\ 31 \\ 83 \end{bmatrix}$$

$c \qquad A^T b$

SOLVE

augmented matrix:

$$\left[\begin{array}{ccc|c} 4 & 6 & 14 & 13 \\ 6 & 14 & 36 & 31 \\ 14 & 36 & 98 & 83 \end{array} \right] \xrightarrow{R_1 \leftarrow R_1/4} \left[\begin{array}{ccc|c} 1 & 1.5 & 3.5 & 3.25 \\ 6 & 14 & 36 & 31 \\ 14 & 36 & 98 & 83 \end{array} \right]$$

$$\begin{array}{l} R_2 \leftarrow R_2 - 6R_1 \\ R_3 \leftarrow R_3 - 14R_1 \end{array} \rightarrow \left[\begin{array}{ccc|c} 1 & 1.5 & 3.5 & 3.25 \\ 0 & 5 & 15 & 11.5 \\ 0 & 15 & 49 & 37.5 \end{array} \right] \xrightarrow{R_2 \leftarrow R_2/5} \left[\begin{array}{ccc|c} 1 & 1.5 & 3.5 & 3.25 \\ 0 & 1 & 3 & 2.3 \\ 0 & 15 & 49 & 37.5 \end{array} \right]$$

$$R_3 \leftarrow R_3 - 15R_2 \rightarrow \left[\begin{array}{ccc|c} 1 & 1.5 & 3.5 & 3.25 \\ 0 & 1 & 3 & 2.3 \\ 0 & 0 & 4 & 3 \end{array} \right] \xrightarrow{R_3 \leftarrow R_3/4} \left[\begin{array}{ccc|c} 1 & 1.5 & 3.5 & 3.25 \\ 0 & 1 & 3 & 2.3 \\ 0 & 0 & 1 & 0.75 \end{array} \right]$$

$c_2 = 0.75$

$$\begin{array}{l} R_2 \leftarrow R_2 - 3R_3 \\ R_1 \leftarrow R_1 - 1.5R_3 \end{array} \rightarrow \left[\begin{array}{ccc|c} 1 & 1.5 & 0 & 0.625 \\ 0 & 1 & 0 & 0.05 \\ 0 & 0 & 1 & 0.75 \end{array} \right] \xrightarrow{R_1 \leftarrow R_1 - 1.5R_2} \left[\begin{array}{ccc|c} 1 & 0 & 0 & 0.55 \\ 0 & 1 & 0 & 0.05 \\ 0 & 0 & 1 & 0.75 \end{array} \right]$$

$c_0 = 0.55$
 $c_1 = 0.05$
 $c_2 = 0.75$

$$y^*(x) = 0.55 + 0.05x + 0.75x^2$$

$$\rho(0.55, 0.05, 0.75) = (0.55 - 0.5)^2 + (0.55 + 0.05 + 0.75 - 1.5)^2 + (0.55 + 2(0.05) + 4(0.75) - 3.5)^2 + (0.55 + 3(0.05) + 9(0.75) - 7.5)^2$$

$$= 0.05$$

since squared error is 0.05,

$$MSE = \rho/N = 0.05/4 = 0.0125$$

Hand work. Analytical solution: parabola.

2.3 Newton–Raphson

One parameter at a time

$$c_j \leftarrow c_j - \frac{\partial \rho / \partial c_j}{\partial^2 \rho / \partial c_j^2} \quad (\text{other parameters frozen})$$

Starting from $c_0 = 0.5$, $c_1 = 1$ (and $c_2 = 0$), $\text{MSE} = 4.25$. Each iteration takes one Newton step on one parameter, then uses the new value while stepping the next. Because ρ is quadratic in each parameter, each step lands exactly on the best value of that parameter given the others.

- **Line** (order c_1 , then c_0 ; curvatures 28 and 4). Iteration 1 gives $(c_0, c_1) = (0.25, 2)$, $\text{MSE} 0.6875$. Iteration 2 gives $(0.0893, 2.1071)$, $\text{MSE} 0.6215$.
- **Parabola** (order c_1, c_2, c_0 ; curvatures 14, 98, 4). Iteration 1 gives $(c_0, c_1, c_2) = (0.1072, 2, 0.0408)$, $\text{MSE} 0.5548$.

The MSE drops each iteration, but slowly, because the parameters are coupled: moving one changes the best value of the others. The code shows the full convergence.

Full multivariate Newton

$$c_{n+1} = c_n - H^{-1}g(c_n)$$

- **Line.** Using the two gradient equations as written, $H = \begin{bmatrix} 4 & 6 \\ 12 & 28 \end{bmatrix}$ and $g(0.5, 1) = (-5, -28)$, so $H^{-1}g = (0.7, -1.3)$ and $c_1 = (-0.2, 2.3)$, $\text{MSE} 0.575$.
- **Parabola.** $H = A^\top A$ from above, $H^{-1} = \frac{1}{20} \begin{bmatrix} 19 & -21 & 5 \\ -21 & 49 & -15 \\ 5 & -15 & 5 \end{bmatrix}$ and $g(0.5, 1, 0) = (-5, -14, -40)$, so $H^{-1}g = (-0.05, 0.95, -0.75)$ and $c_1 = (0.55, 0.05, 0.75)$, $\text{MSE} 0.0125$.

In both cases the gradient at the new point is zero, so iteration 2 changes nothing. One step is enough because Newton fits a quadratic model, and ρ is already exactly quadratic; the result is the same as the analytical solution.

Numerical method (NR multivariate)

one parameter at a time ~~this is not the full multivariate~~. That is below.

For a line

UPDATERULE: $x_{\text{new}} = x - \frac{f(x)}{f'(x)}$ where $f(x)$ is first derivative of P (what we want to minimize)

$$P = (C_0 - 0.5)^2 + (C_0 + C_1 - 1.5)^2 + (C_0 + 2C_1 - 3.5)^2 + (C_0 + 3C_1 - 7.5)^2$$

take partial derivatives: $(\frac{\partial P}{\partial C_0}, \frac{\partial P}{\partial C_1})$ as done above.

$$\textcircled{1} \quad 4C_0 + 6C_1 - 13 = 0 \quad \leftarrow \frac{\partial P}{\partial C_0}$$

$$\textcircled{2} \quad 12C_0 + 28C_1 - 62 = 0 \quad \leftarrow \frac{\partial P}{\partial C_1}$$

find second derivatives (curvature)

$$\frac{\partial^2 P}{\partial C_0^2} = 4, \quad \frac{\partial^2 P}{\partial C_1^2} = 28$$

initial guess: $C_0 = 0.5, C_1 = 1 \Rightarrow \text{MSE} = 4.25$ (sub C values into eqn and mean)

ITERATION ONE

update C_1 (keep C_0 unchanged @ 0.5)

$$\frac{\partial P}{\partial C_1} = 12C_0 + 28C_1 - 62 = 0$$

$$\Rightarrow 12(0.5) + 28(1) - 62 = -28$$

* slope is negative so increase C_1

$$\text{then } C_{1, \text{new}} = 1 - (-28/28) = 2$$

update C_0 (keep C_1 unchanged @ 2) new value

$$\frac{\partial P}{\partial C_0} = 4C_0 + 6C_1 - 13 = 0$$

$$\Rightarrow 4(0.5) + 6(2) - 13 = 1$$

* slope is positive so decrease C_0

$$\text{then } C_{0, \text{new}} = 0.5 - (1/4) = 0.25$$

with these new C_0, C_1 values after 1 iteration we get $\text{MSE} = 0.6875$

which is down from 4.25 but still far from converging.

ITERATION TWO

update C_1 (freeze $C_0 = 0.25$)

$$\Rightarrow 12(0.25) + 28(1) - 62 = -3$$

* slope is -ve so increase C_1

$$C_{1, \text{new}} = 2 - (-3/28) = 2.1071$$

update C_0 (freeze $C_1 = 2.1071$)

$$\Rightarrow 4(0.25) + 6(2.1071) - 13 = 0.642$$

$$C_{0, \text{new}} = 0.25 - (0.642/4) = 0.0895$$

New MSE with C_0, C_1 plugged in is 0.6215 so improving but still not converged.

See correct answer with code. For hand we stop at iteration 2.

For a parabola

$$y = C_0 + C_1 x + C_2 x^2$$

slopes:

$$\frac{\partial P}{\partial C_0} = 4C_0 + 6C_1 + 14C_2 = 13 = g_0$$

$$\frac{\partial P}{\partial C_1} = 6C_0 + 14C_1 + 36C_2 = 31 = g_1$$

$$\frac{\partial P}{\partial C_2} = 14C_0 + 36C_1 + 98C_2 = 83 = g_2$$

Second derivatives:

$$\frac{\partial^2 P}{\partial C_0^2} = 4$$

$$\frac{\partial^2 P}{\partial C_1^2} = 14$$

$$\frac{\partial^2 P}{\partial C_2^2} = 98$$

initial guess: $C_0 = 0.5, C_1 = 1, C_2 = 0$, then starting $\text{MSE} = 4.25$

Hand work. One parameter at a time: line, and parabola setup.

ITERATION ONE

update c_1 (freeze $c_0 = 0.5$, $c_2 = 0$)

$$g_1 = 6(0.5) + 14(1) + 36(0) - 31 = -14$$

$$c_1 = 1 - (-14/14) = 2$$

update c_2 (freeze $c_0 = 0.5$, $c_1 = 2$)

$$g_2 = 14(0.5) + 36(2) + 98(0) - 83 = -4$$

$$c_2 = 0 - (-4/98) = 0.0408$$

update c_0 (freeze $c_1 = 2$, $c_2 = 0.0408$)

$$g_0 = 4(0.5) + 6(2) + 14(0.0408) - 13 = 1.5712$$

$$c_0 = 0.5 - (1.5712/4) = 0.1072$$

MSE = 0.5548, closer but not converged. (far from it)

3 params strongly affect one another. see true correct answer with code. Will require lots of iterations. See here just the process of iterations.

Full NR Multivariate

For a line

$$x_{n+1} = x_n - H_p^{-1}(x) g_p(x_n)$$

$$H = \begin{bmatrix} 4 & 6 \\ 12 & 28 \end{bmatrix} \leftarrow \text{the hessian for the line}$$

$$\det H = 4(28) - 6(12) = 40$$

$$H^{-1} = \frac{1}{40} \begin{bmatrix} 28 & -6 \\ -12 & 4 \end{bmatrix}$$

$$\left\{ \begin{array}{l} \textcircled{1} \quad 4c_0 + 6c_1 - 13 = 0 \quad \leftarrow \partial p / c_0 \\ \textcircled{2} \quad 12c_0 + 28c_1 - 62 = 0 \quad \leftarrow \partial p / c_1 \end{array} \right.$$

And second derivatives (curvature)

$$\frac{\partial^2 p}{\partial c_0^2} = 4, \quad \frac{\partial^2 p}{\partial c_1^2} = 6$$

$$\frac{\partial^2 p}{\partial c_0 \partial c_1} = 12, \quad \frac{\partial^2 p}{\partial c_1 \partial c_0} = 28$$

ITERATION ONE

the gradient at $(0.5, 1)$ is $\nabla p = (-5, -28)$

$$H^{-1} g_0 = \frac{1}{40} \begin{bmatrix} 28 & -6 \\ -12 & 4 \end{bmatrix} \begin{bmatrix} -5 \\ -28 \end{bmatrix} = \frac{1}{40} \begin{bmatrix} 28 \\ -52 \end{bmatrix} = \begin{bmatrix} 0.7 \\ -1.3 \end{bmatrix}$$

apply formula $x_1 = x_0 - H^{-1} g_0$:

$$x_1 = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} - \begin{bmatrix} 0.7 \\ -1.3 \end{bmatrix} = \begin{bmatrix} -0.2 \\ 2.3 \end{bmatrix}$$

then sub into p to find MSE is 0.575

ITERATION TWO

$$x_1 = \begin{bmatrix} -0.2 \\ 2.3 \end{bmatrix} - 0 = \begin{bmatrix} -0.2 \\ 2.3 \end{bmatrix} \quad \text{nothing changed with MSE} = 0.575.$$

so solution converged.

For a parabola:

$$H = \begin{bmatrix} 4 & 6 & 14 \\ 6 & 14 & 36 \\ 14 & 36 & 98 \end{bmatrix}$$

find inverse of Hessian:

$$\Rightarrow H^{-1} = \frac{1}{20} \begin{bmatrix} 19 & -21 & 5 \\ -21 & 49 & -15 \\ 5 & -15 & 5 \end{bmatrix}$$

ITERATION ONE:

$$g_0 = 2 + 6 + 0 - 13 = -5$$

$$g_1 = 3 + 14 + 0 - 31 = -14$$

$$g_2 = 7 + 36 + 0 - 83 = -40$$

Hand work. Parabola iteration; full Newton for the line.

$$H^{-1}g = \frac{1}{20} \begin{bmatrix} 19 & -21 & 5 \\ -21 & 49 & -15 \\ 5 & -15 & 5 \end{bmatrix} \begin{bmatrix} -5 \\ -14 \\ -40 \end{bmatrix} = \frac{1}{20} \begin{bmatrix} -1 \\ 19 \\ -15 \end{bmatrix} = \begin{bmatrix} -0.05 \\ 0.95 \\ -0.75 \end{bmatrix}$$

$$x_1 = \begin{bmatrix} 6.5 \\ 1 \\ 0 \end{bmatrix} - \begin{bmatrix} -0.05 \\ 0.95 \\ -0.75 \end{bmatrix} = \begin{bmatrix} 6.55 \\ 0.05 \\ 0.75 \end{bmatrix}$$

So $MSE = 0.0125$ (sub in these values into p and mean)

ITERATION TWO

$$x_1 = \begin{bmatrix} 6.55 \\ 0.05 \\ 0.75 \end{bmatrix} \quad \text{where gradient at } x_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{so } H^{-1} \cdot 0 = 0 \Rightarrow x_{\text{new}} = x_1$$

So the method has converged with $MSE = 0.0125$

makes sense given NR is a quadratic approximation and p is already exactly quadratic.

Hand work. Full Newton for the parabola.

Automating the fit in Python

2.4 Code

`fitting.py` builds the design matrix A with columns $1, x, x^2$, so that $\hat{y} = Ac$. It then uses $\text{MSE} = \frac{1}{N} \|Ac - y\|^2$, gradient $\frac{2}{N} A^\top (Ac - y)$ and Hessian $\frac{2}{N} A^\top A$. It solves:

- **Analytically:** the normal equations, by Gauss–Jordan elimination (the same row operations as by hand).
- **Newton one parameter at a time:** with the same start and update order as the hand work, until no parameter moves more than 10^{-10} .
- **Full multivariate Newton.**

It prints the first two iterations so they can be checked against the hand work. Run `python fitting.py` from `A1/code/`.

fitting.py

Python

```
"""
SYDE 572 Assignment 1, Part 2: fit a line and a parabola to 4 points by minimizing the MSE.

    analytical      : solve the normal equations (A^T A) c = A^T y by Gauss-Jordan
elimination
    one at a time   : Newton-Raphson on one parameter, the others frozen (the hand method)
    full Newton     : c ← c - H^-1 g on all parameters at once

    python fitting.py      (plots are saved to ../media/)
"""

import os

import numpy as np

import plots

MEDIA = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "media")

X = np.array([0.0, 1.0, 2.0, 3.0])
Y = np.array([0.5, 1.5, 3.5, 7.5])

def design_matrix(x, degree):
    """Columns 1, x, x^2, ... so the model is y_hat = A @ c with c = [c0, c1, ...]."""
    return np.vander(x, degree + 1, increasing=True)
```

```

def mse(A, y, c):
    return np.mean((A @ c - y) ** 2)

def gradient(A, y, c):
    return 2 / len(y) * A.T @ (A @ c - y)

def hessian(A):
    return 2 / len(A) * A.T @ A          # constant, because the MSE is quadratic in c

def gauss_jordan(M, b):
    """Solve M c = b with the same row operations as the hand work."""
    M = np.column_stack([M, b]).astype(float)
    for i in range(len(b)):
        M[i] /= M[i, i]
        for k in range(len(b)):
            if k != i:
                M[k] -= M[k, i] * M[i]
    return M[:, -1]

def newton_one_at_a_time(A, y, c_start, order, tol=1e-10, max_iter=5000):
    """Each iteration updates the parameters in `order`, one at a time:
    c_j ← c_j - g_j / H_jj with the others frozen. Returns every intermediate c."""
    c = np.array(c_start, float)
    H = hessian(A)
    path = [c.copy()]
    for _ in range(max_iter):
        before = c.copy()
        for j in order:
            c[j] -= gradient(A, y, c)[j] / H[j, j]
            path.append(c.copy())
        if np.max(np.abs(c - before)) < tol:
            break
    return c, np.array(path)

def newton_full(A, y, c_start, tol=1e-10, max_iter=20):
    c = np.array(c_start, float)
    path = [c.copy()]
    for _ in range(max_iter):
        step = gauss_jordan(hessian(A), gradient(A, y, c))
        c -= step
        path.append(c.copy())
        if np.max(np.abs(step)) < tol:
            break
    return c, np.array(path)

# (name, degree, starting guess, update order) -- same start and order as the hand work
MODELS = [

```

```

("line", 1, [0.5, 1.0], [1, 0]),
("parabola", 2, [0.5, 1.0, 0.0], [1, 2, 0]),
]

if __name__ == "__main__":
    os.makedirs(MEDIA, exist_ok=True)
    plots.setup_style()
    finals = []
    for name, degree, start, order in MODELS:
        A = design_matrix(X, degree)
        loss = lambda c, A=A: mse(A, Y, c)
        c_exact = gauss_jordan(A.T @ A, A.T @ Y)
        c_one, one_path = newton_one_at_a_time(A, Y, start, order)
        c_full, full_path = newton_full(A, Y, start)
        n_iters = (len(one_path) - 1) // len(order)

        print(f"\n{name.upper()} (start {start}, MSE {loss(np.array(start)):.4f})")
        print(f" analytical c = {np.round(c_exact, 6)} MSE = {loss(c_exact):.6f}")
        for k in (1, 2):
            c_k = one_path[k * len(order)]
            print(f" one-at-a-time iter {k}: c = {np.round(c_k, 4)} MSE = {loss(c_k):.4f}")
            print(f" one-at-a-time c = {np.round(c_one, 6)} MSE = {loss(c_one):.6f} ({n_iters} iterations)")
            print(f" full Newton c = {np.round(c_full, 6)} MSE = {loss(c_full):.6f} ({len(full_path) - 2} step + 1 check)")

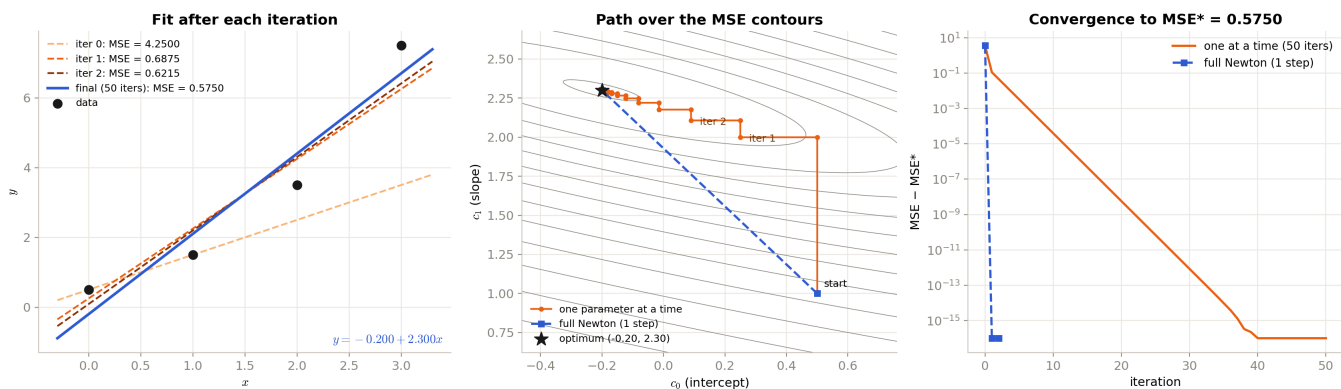
        plots.plot_fit_steps(name, X, Y, loss, one_path, full_path, c_exact,
                             os.path.join(MEDIA, f"p2_{name}_steps.png"))
        finals.append((name.capitalize(), c_exact, loss(c_exact)))

    plots.plot_final_fits(X, Y, finals, os.path.join(MEDIA, "p2_final_fits.png"))

```

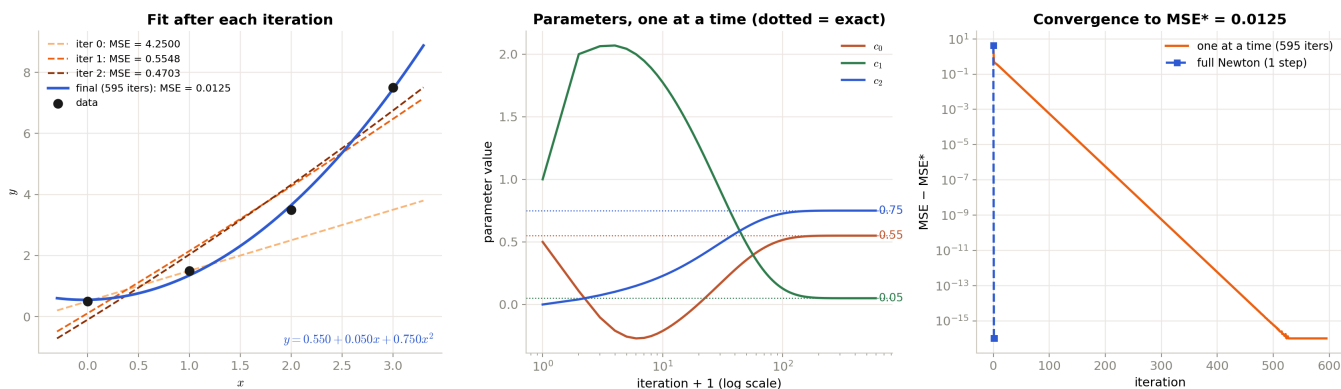
2.5 Intermediate steps

Fitting a line: Newton-Raphson intermediate steps



Line. Left: the fit after iterations 0–2 and the final fit. Middle: one-parameter-at-a-time Newton moves along one axis at a time, zig-zagging across the tilted MSE contours; full Newton goes straight to the optimum. Right: the excess MSE falls in a straight line on a log scale (linear convergence).

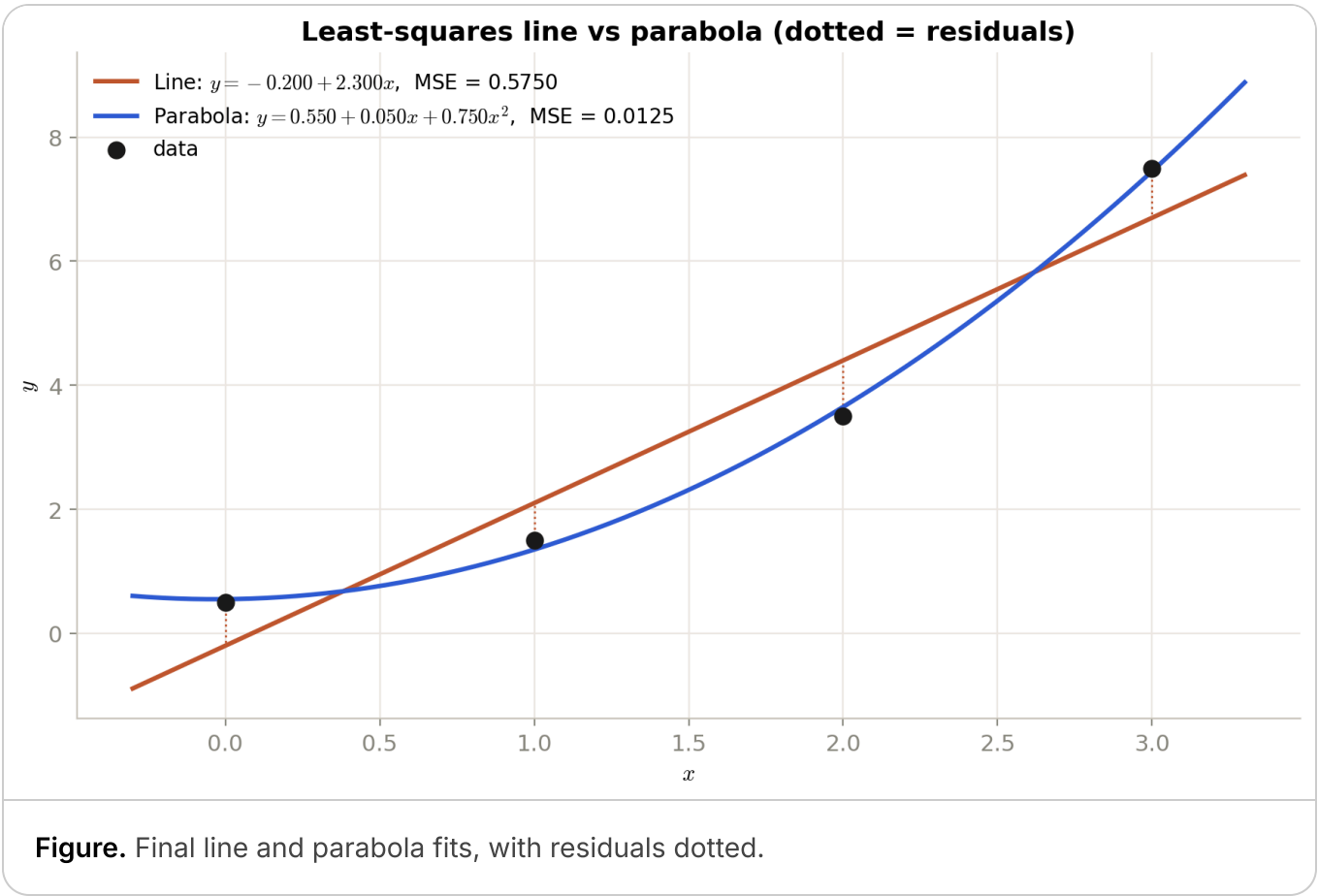
Fitting a parabola: Newton-Raphson intermediate steps



Parabola. Middle: each parameter over the iterations. c_1 first overshoots to about 2 (trying to act as a line) and only drifts back once c_2 grows, which is why it takes 595 iterations.

2.6 Results

Model	Method	c_0	c_1	c_2	Iterations	MSE
Line	Analytical	-0.2	2.3	—	—	0.575
Line	Newton, one at a time	-0.2	2.3	—	50	0.575
Line	Newton, full	-0.2	2.3	—	1	0.575
Parabola	Analytical	0.55	0.05	0.75	—	0.0125
Parabola	Newton, one at a time	0.55	0.05	0.75	595	0.0125
Parabola	Newton, full	0.55	0.05	0.75	1	0.0125



Discussion

The MSE is **0.575 for the line** and **0.0125 for the parabola**. The parabola fits 46× better because the data curve upward: the line's residuals follow a pattern (points above the line at both ends, below it in the middle), which a straight line can't remove. Every method reaches the same parameters. Full Newton matches the analytical answer in one step because the MSE is exactly quadratic. One-at-a-time Newton converges only linearly, and more slowly the more coupled the parameters are: 50 iterations for the line, 595 for the parabola.

References & acknowledgements

- SYDE 572 Assignment 1 handout (Newton–Raphson and golden section search code).
- Cardano's formula for the real root of a depressed cubic $x^3 + px + q = 0$.
- Plots made with NumPy and Matplotlib.